Practical 1: Types of forecasting – Qualitative and Quantitative

Program:

R Program for Qualitative Forecasting (Expert Ratings)

Real-Time Application
Before launching a new mobile phone, a company such as Samsung Electronics may gather forecasts from marketing experts and sales managers to estimate initial demand

Problem Statement
Collect expert ratings and calculate average demand prediction.

```
expert_opinions <- c(500, 550, 520, 530, 540)

# Average forecast
forecast <- mean(expert_opinions)

print(paste("Forecasted Demand:", forecast))
```

Output

Forecasted Demand: 528

Code Explanation:

1. Create a Vector of Expert Opinions

expert_opinions <- c(500, 550, 520, 530, 540)

- `expert_opinions` is a variable.
- `<-` is the assignment operator used to store values.
- `c()` stands for **combine**.
- It combines five expert estimates into a single vector.

Expert	Demand Estimate
1	500
2	550
3	520

4 530

5 540

After execution:

expert_opinions

Output:

[1] 500 550 520 530 540

---

2. Calculate the Average Forecast

forecast <- mean(expert_opinions)

- `mean()` calculates the arithmetic average.
- It takes all values from `expert_opinions`.

Calculation:

$$\text{Mean} = \frac{500+550+520+530+540}{5} = \frac{2640}{5} = 528$$

So,

forecast = 528

---

3. Display the Result

print(paste("Forecasted Demand:", forecast))

`paste()`

Combines text and values into a single string.

paste("Forecasted Demand:", forecast)

Output:

"Forecasted Demand: 528"

`print()`

Displays the result on the console.

Final Output:

[1] "Forecasted Demand: 528"

==========

2. Quantitative Forecasting (Moving Average Method)

Real-Time Scenario

A retail store has sales data for the last 6 months and wants to forecast next month's sales.

Month	Sales
Jan	120
Feb	135
Mar	150
Apr	145
May	160
Jun	170

R Program

```
# Historical sales data
sales <- c(120, 135, 150, 145, 160, 170)

# 3-month moving average forecast
forecast <- mean(tail(sales, 3))

print(paste("Forecast for Next Month:", forecast, "units\n"))
```

Output

Forecast for Next Month: 158.3333 units

Code Explanation:

.Line 1: Create Historical Sales Data

sales <- c(120, 135, 150, 145, 160, 170)

Meaning

- `sales` is a variable that stores monthly sales.
- `<-` is the assignment operator.
- `c()` means **combine** values into a vector.

Stored data:

Month	Sales
January	120
February	135
March	150
April	145
May	160
June	170

Output if you type:

sales
[1] 120 135 150 145 160 170

Line 2: Calculate Moving Average Forecast

forecast <- mean(tail(sales, 3))

This line contains two functions:

A. `tail(sales, 3)`

- `tail()` returns values from the end of a vector.
- `3` means take the last 3 values.

tail(sales, 3)

Output:

[1] 145 160 170

These represent:

- April = 145
- May = 160
- June = 170

B. `mean()`

mean(145,160,170)

`mean()` calculates the average.

Formula:

Average=Sum of ValuesNumber of Values\text{Average} = \frac{\text{Sum of Values}}{\text{Number of Values}}Average=Number of ValuesSum of Values

Calculation:

145+160+1703\frac{145 + 160 + 170}{3}3145+160+170 4753\frac{475}{3}3475 158.33158.33158.33

So:

forecast = 158.33

Line 3: Display Result

print(paste("Forecast for Next Month:", forecast, "units\n"))

`paste()`

Combines text and values.

paste("Forecast for Next Month:", forecast, "units")

Result:

"Forecast for Next Month: 158.3333 units"

`print()`

Displays the result on the screen.

Output:

[1] "Forecast for Next Month: 158.3333 units"

Practical 2: Data collection, classification and presentation

Program:

```
student_id <- c(101,102,103,104,105,106,107,108,109,110)

student_name <- c("Amit","Priya","Rahul","Sneha",
                  "Vikas","Neha","Ajay","Pooja",
                  "Rohan","Anjali")

marks <- c(85,72,91,65,78,88,55,92,70,81)

student_data <- data.frame(student_id,
                           student_name,
                           marks)

student_data$grade <- ifelse(student_data$marks >= 80, "A",
                      ifelse(student_data$marks >= 60,
                             "B","C"))

print(student_data)

summary(student_data$marks)

grade_count <- table(student_data$grade)

print(grade_count)

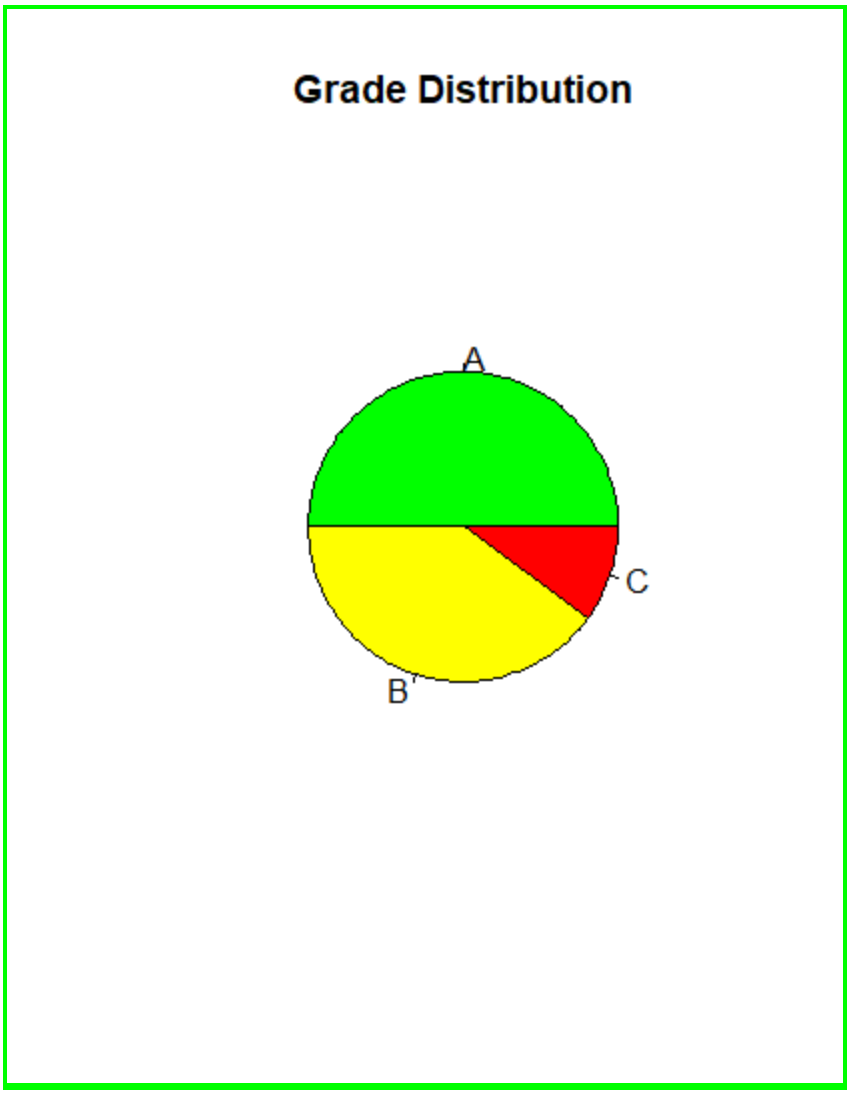
barplot(grade_count,
        main="Grade Distribution",
        col=c("green","yellow","red"))

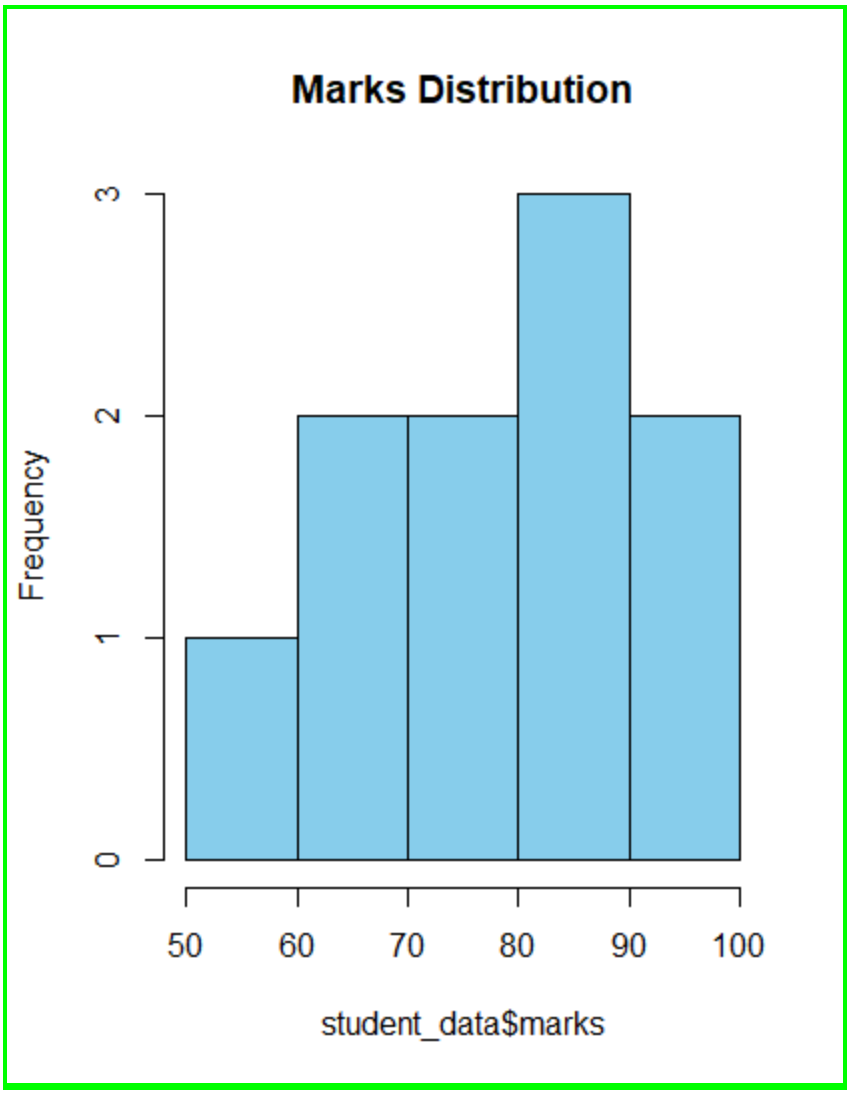
pie(grade_count,
    main="Grade Distribution",
    col=c("green","yellow","red"))

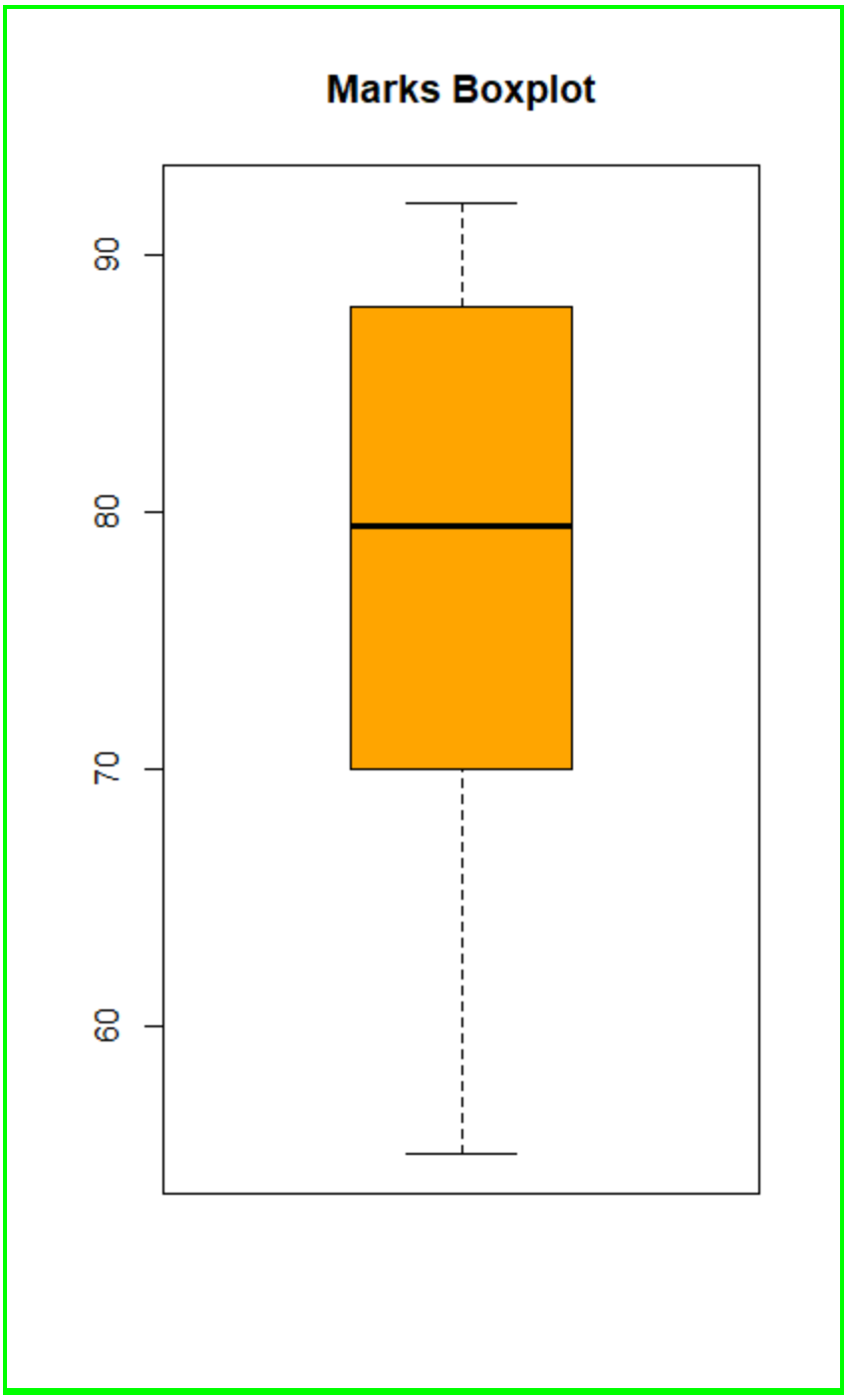
hist(student_data$marks,
     main="Marks Distribution",
     col="skyblue")

boxplot(student_data$marks,
        main="Marks Boxplot",
        col="orange")
```

Output


Grade Distribution
A
C
B


Marks Distribution
Frequency
0
1
2
3
50
60
70
80
90
100
student_data$marks


Marks Boxplot
90
80
70
60

Explanation:

Real-Time Example: Student Marks Analysis

Suppose a college collected marks of students in a Data Science subject.

Step 1: Data Collection

R Code

```
# Student Marks Data Collection

student_id <- c(101,102,103,104,105,106,107,108,109,110)

student_name <- c("Amit","Priya","Rahul","Sneha",
                  "Vikas","Neha","Ajay","Pooja",
                  "Rohan","Anjali")

marks <- c(85,72,91,65,78,88,55,92,70,81)

student_data <- data.frame(student_id,
                           student_name,
                           marks)

print(student_data)
```

Output

Student ID	Name	Marks
101	Amit	85
102	Priya	72
103	Rahul	91
104	Sneha	65
105	Vikas	78
106	Neha	88
107	Ajay	55
108	Pooja	92

109	Rohan	70
110	Anjali	81

Step 2: Data Classification

Classify students according to grades.

Grade Criteria

Marks	Grade
80 and above	A
60-79	B
Below 60	C

R Code

```
student_data$grade <- ifelse(student_data$marks >= 80, "A",
                      ifelse(student_data$marks >= 60,
                        "B","C"))

print(student_data)
```

Output

Name	Marks	Grade
Amit	85	A
Priya	72	B
Rahul	91	A
Sneha	65	B
Vikas	78	B
Neha	88	A
Ajay	55	C
Pooja	92	A
Rohan	70	B

Anjali 81 A

Step 3: Data Presentation

A. Summary Statistics

R Code

```
summary(student_data$marks)
```

Output

```
Min.   :55
1st Qu.:70
Median :79.5
Mean   :77.7
3rd Qu.:87
Max.   :92
```

B. Frequency Distribution

R Code

```
table(student_data$grade)
```

Output

```
A  B  C
5  4  1
```

C. Bar Chart

R Code

```
grade_count <- table(student_data$grade)

barplot(grade_count,
      main="Grade Distribution",
      xlab="Grade",
      ylab="Number of Students",
      col=c("green","yellow","red"))
```

Interpretation

- Grade A = 5 Students

- Grade B = 4 Students
- Grade C = 1 Student

D. Pie Chart

R Code

```
pie(grade_count,
  main="Student Grade Distribution",
  col=c("green","yellow","red"))
```

Interpretation

Pie chart shows percentage distribution of grades.

E. Histogram

R Code

```
hist(student_data$marks,
   main="Distribution of Marks",
   xlab="Marks",
   col="skyblue",
   border="black")
```

Interpretation

Histogram shows how marks are distributed among students.

F. Box Plot

R Code

```
boxplot(student_data$marks,
      main="Student Marks Boxplot",
      col="orange")
```

Interpretation

- Shows minimum marks.
- Maximum marks.
- Median marks.
- Outliers (if any).

Practical 3

Introduction to Excel and Statistical Software for Forecasting

1. Introduction

Forecasting is the process of predicting future values based on historical data. Organizations use forecasting to estimate sales, demand, profits, stock prices, weather conditions, and business growth.

Statistical software and spreadsheet tools help analyze past data and generate accurate forecasts.

A. Microsoft Excel

Definition

Microsoft Excel is a spreadsheet application developed by Microsoft that allows users to organize, analyze, calculate, visualize, and forecast data using formulas, functions, charts, and statistical tools.

Features of Excel

1. Data Entry and Storage

- Stores data in rows and columns.
- Supports large datasets.

2. Mathematical Calculations

- Performs arithmetic operations.
- Uses formulas and functions.

Example:

=SUM(A1:A10)

=AVERAGE(B1:B10)

=MAX(C1:C10)

3. Data Analysis

- Sorting and filtering.
- Pivot Tables.
- Statistical functions.

4. Data Visualization

- Bar Charts
- Pie Charts
- Line Charts
- Histograms

5. Forecasting Tools

- Trend Analysis
- Moving Average
- Forecast Sheet
- Regression Analysis

Forecasting in Excel

Example: Monthly Sales Forecast

Month	Sales
Jan	100
Feb	120
Mar	130
Apr	150
May	170

Forecast Formula

=FORECAST.LINEAR(6,B2:B6,A2:A6)

Output:

Predicted Sales = 185

This predicts sales for Month 6 based on previous sales data.

Advantages of Excel

- Easy to learn.
- User-friendly interface.
- Built-in forecasting tools.
- Widely used in business.
- No programming knowledge required.

Limitations of Excel

- Not suitable for very large datasets.
- Limited advanced machine learning capabilities.
- Slower than specialized statistical software.

B. Statistical Software for Forecasting

Definition

Statistical software consists of specialized programs used for data analysis, statistical modeling, forecasting, and predictive analytics.

Popular Statistical Software

Software	Purpose

R	Statistical analysis and forecasting
SPSS	Business and research analytics
SAS	Enterprise analytics
Minitab	Quality control and forecasting
Stata	Econometrics and forecasting
Python	Machine learning and forecasting

Forecasting Techniques

1. Moving Average

Forecast based on average of previous observations.

Formula:

Moving Average=∑XnMoving\ Average = \frac{\sum X}{n}Moving Average=n∑X

Example:

Sales = 100, 120, 130

Forecast:

(100+120+130)/3=116.67(100+120+130)/3 = 116.67(100+120+130)/3=116.67

---

2. Linear Regression

Predicts future values using relationship between variables.

Equation:

Y=a+bXY = a + bXY=a+bX

Where:

- Y = Predicted value
- X = Independent variable
- a = Intercept
- b = Slope

Applications:

- Sales forecasting
- Demand prediction
- Market analysis

---

3. Time Series Forecasting

Uses historical observations recorded over time.

Examples:

- Monthly sales
- Temperature records
- Stock prices

Methods:

- Moving Average
- Exponential Smoothing
- ARIMA Models

Forecasting in R

Example

```
sales <- c(100,120,130,150,170)



time <- c(1,2,3,4,5)



model <- lm(sales ~ time)
```

```
predict(model,
    data.frame(time=6))
```

Output:

185

Predicted sales for Month 6 = 185

Applications of Forecasting

Business

- Sales prediction
- Revenue forecasting
- Inventory management

Banking

- Loan demand forecasting
- Risk assessment

Healthcare

- Disease outbreak prediction
- Patient admission forecasting

Education

- Student enrollment prediction
- Performance analysis

Weather

- Rainfall forecasting
- Temperature prediction

Comparison: Excel vs Statistical Software

Feature	Excel	Statistical Software

Ease of Use	High	Moderate
Programming Required	No	Sometimes
Large Data Handling	Limited	Excellent
Forecasting Accuracy	Moderate	High
Machine Learning Support	Limited	Advanced
Cost	Usually Available	Some are expensive

Practical 4:

Components of Time Series

Program:

```
# Monthly Sales Data (24 Months)


sales <- c(
  100,120,140,180,200,220,240,230,210,260,280,320,
  110,130,150,190,210,240,260,250,230,280,300,340
)


# Create Time Series Object
sales_ts <- ts(sales,
               start = c(2024,1),
               frequency = 12)


# Display Time Series
print(sales_ts)


# Plot Original Series
plot(sales_ts,
     main = "Monthly Sales Time Series",
     xlab = "Month",
     ylab = "Sales",
     col = "blue",
     lwd = 2)


# Decompose Time Series
decomp <- decompose(sales_ts)
```

```
# Plot Components
plot(decomp)

# Extract Components
trend <- decomp$trend
seasonal <- decomp$seasonal
random <- decomp$random

# Print Components
cat("Trend Component:\n")
print(trend)

cat("\nSeasonal Component:\n")
print(seasonal)

cat("\nRandom Component:\n")
print(random)

# Moving Average Trend
trend_ma <- filter(sales_ts,
                   rep(1/3,3),
                   sides = 2)

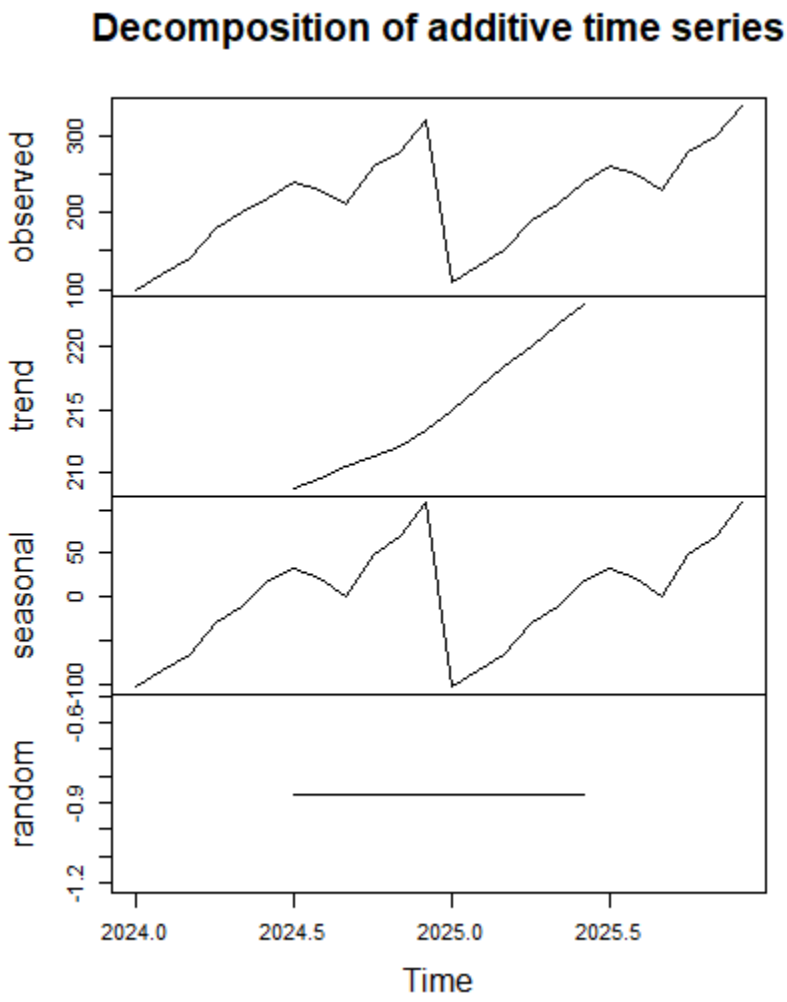
# Plot Trend
plot(sales_ts,
     main = "Trend Analysis using Moving Average",
     xlab = "Month",
```

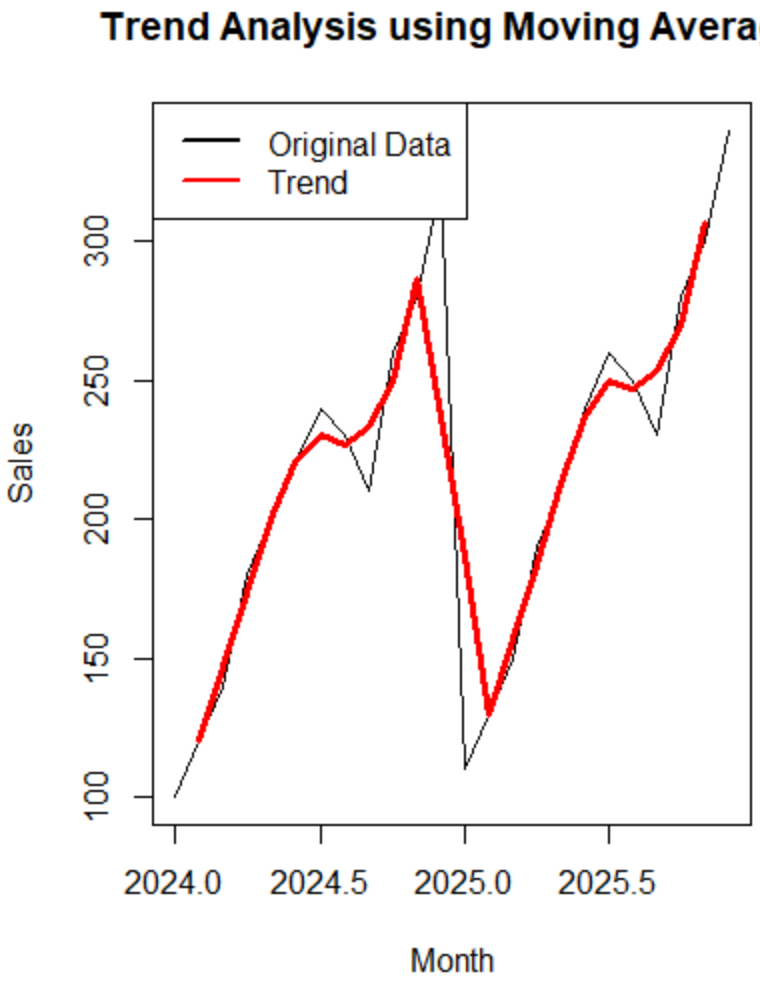
```
      ylab = "Sales")


lines(trend_ma,
      col = "red",
      lwd = 3)


legend("topleft",
       legend = c("Original Data","Trend"),
       col = c("black","red"),
       lty = 1,
       lwd = 2)
```

Output:





Explanation

Components of Time Series with R Programming (Real-Time Example)

What is Time Series?

A Time Series is a sequence of observations recorded at regular intervals of time such as daily, monthly, quarterly, or yearly.

Examples

- Monthly sales of a supermarket
- Daily stock prices
- Annual rainfall
- Monthly electricity consumption
- Quarterly company profits

Components of Time Series

A Time Series consists of four major components:

1. Trend (T)

The long-term movement of data over time.

Example

Monthly sales increasing every year due to business growth.

Year	Sales
2021	1000
2022	1200
2023	1500
2024	1800

Trend = Upward

---

2. Seasonal Variation (S)

Regular patterns that repeat within a fixed period.

Example

Ice cream sales increase every summer and decrease during winter.

Characteristics

- Repeats every month, quarter, or season.

- Predictable pattern.

---

3. Cyclical Variation (C)

Long-term fluctuations occurring over several years.

Example

Economic boom and recession cycles.

Characteristics

- Duration greater than one year.
- Not fixed like seasonal variation.

---

4. Irregular Variation (I)

Unexpected random changes.

Example

- COVID-19 pandemic
- Floods
- Earthquakes
- Strikes

These cannot be predicted accurately.

Time Series Model

Additive Model

$Y=T+S+C+IY = T + S + C + IY=T+S+C+I$

Where:

- T = Trend
- S = Seasonal
- C = Cyclical
- I = Irregular

Real-Time Example

Suppose a mobile shop records monthly sales (in ₹ thousand).

Month	Sales
Jan	100
Feb	120
Mar	140
Apr	180
May	200
Jun	220
Jul	240
Aug	230
Sep	210
Oct	260
Nov	280
Dec	320

Step 1: Create Time Series Data

```
sales <- c(100,120,140,180,200,220,
           240,230,210,260,280,320)


sales_ts <- ts(sales,
               start=c(2025,1),
               frequency=12)


print(sales_ts)
```

Step 2: Plot Time Series

```
plot(sales_ts,
   main="Monthly Mobile Sales",
   xlab="Month",
   ylab="Sales (Thousands)",
   col="blue",
   lwd=2)
```

Interpretation

- Overall sales are increasing.
- Indicates an upward trend.

Step 3: Identify Trend Component

```
trend <- decompose(sales_ts)
```

```
plot(trend)
```

Output displays:

1. Original Series
2. Trend
3. Seasonal Component
4. Random Component

Step 4: Moving Average Trend

```
trend_ma <- filter(sales_ts,

                   rep(1/3,3),

                   sides=2)



plot(sales_ts,

   main="Trend Analysis")



lines(trend_ma,

    col="red",

    lwd=3)
```

Interpretation

Red line shows the trend.

Step 5: Seasonal Component

```
decomp <- decompose(sales_ts)



seasonal <- decomp$seasonal



print(seasonal)
```

Interpretation

Positive values indicate high seasonal demand.

Negative values indicate low seasonal demand.

Step 6: Random Component

random <- decomp$random

print(random)

Interpretation

Random fluctuations due to unexpected events.

##########################

Practical 5

1. Simple Moving Average (SMA)

```
# Simple Moving Average (SMA)

# Input data
data <- c(10, 20, 30, 40, 50, 60, 70, 80)

# Window size
n <- 3

# Calculate Simple Moving Average
sma <- filter(data, rep(1/n, n), sides = 1)

# Display results
print("Original Data:")
print(data)

print("Simple Moving Average:")
print(sma)
```

Output

```
Original Data:
10 20 30 40 50 60 70 80

Simple Moving Average:
NA NA 20 30 40 50 60 70
```

---

2. Weighted Moving Average (WMA)

```
# Weighted Moving Average (WMA)

# Input data
data <- c(10, 20, 30, 40, 50, 60, 70, 80)

# Weights
weights <- c(1, 2, 3)

# Length of weights
n <- length(weights)

# Create empty vector
wma <- rep(NA, length(data))

# Calculate Weighted Moving Average
```

```
for(i in n:length(data))
{
  wma[i] <- sum(data[(i-n+1):i] * weights) / sum(weights)
}

# Display results
print("Original Data:")
print(data)

print("Weighted Moving Average:")
print(wma)
```

Output

Original Data:
10 20 30 40 50 60 70 80

Weighted Moving Average:
NA NA 23.33 33.33 43.33 53.33 63.33 73.33

Practical 6

Trend analysis – Linear trend method

Business Scenario

A food delivery startup wants to predict the number of daily deliveries to hire delivery partners.

Data

Month	Deliveries
Jan	8200
Feb	8500
Mar	8900
Apr	9300
May	9700
Jun	10100

Tasks

1. Apply the Linear Trend Method.
2. Find the trend line.
3. Predict deliveries for July.
4. Suggest whether additional riders should be hired.

Code

```
# ----------------------------------------
# Practical: Linear Trend Analysis in R
# Business Scenario:
# Predict food deliveries for July
# ----------------------------------------

# Step 1: Create the dataset
Month <- c("Jan", "Feb", "Mar", "Apr", "May", "Jun")
Deliveries <- c(8200, 8500, 8900, 9300, 9700, 10100)

# Step 2: Assign X values (Time)
X <- 1:6

# Step 3: Create Data Frame
delivery_data <- data.frame(Month, X, Deliveries)

# Display Dataset
print("Food Delivery Dataset")
print(delivery_data)

# Step 4: Apply Linear Trend Method
trend_model <- lm(Deliveries ~ X)

# Step 5: Display Trend Equation
print("Linear Trend Model")
print(summary(trend_model))

# Extract coefficients
a <- coef(trend_model)[1]
b <- coef(trend_model)[2]

cat("\nTrend Equation:\n")
cat("Y =", round(a,2), "+", round(b,2), "* X\n")

# Step 6: Predict Deliveries for July (X = 7)
july_prediction <- predict(trend_model,
                           newdata = data.frame(X = 7))

cat("\nPredicted Deliveries for July =",
    round(july_prediction), "\n")

# Step 7: Plot Actual Data and Trend Line
plot(X, Deliveries,
     pch = 19,
     col = "blue",
     xlab = "Month Number",
     ylab = "Daily Deliveries",
     main = "Linear Trend Analysis")
```

```
abline(trend_model,
       col = "red",
       lwd = 2)

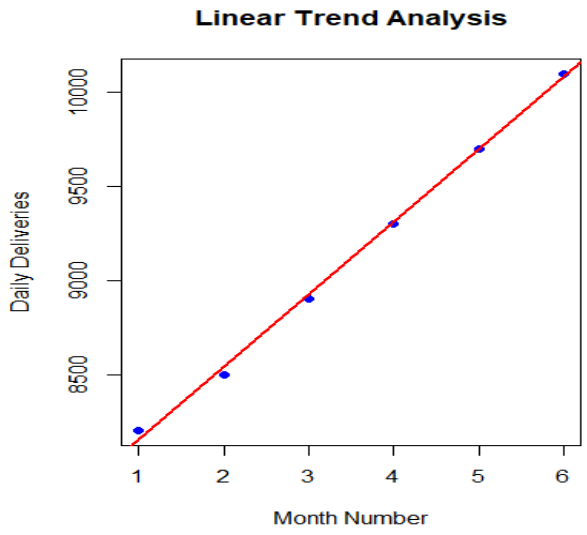
# Add July Prediction
points(7, july_prediction,
       pch = 19,
       col = "green",
       cex = 1.5)

text(7, july_prediction,
     labels = "July Forecast",
     pos = 3,
     col = "green")

# Step 8: Business Decision
if(july_prediction > max(Deliveries)){
  cat("\nBusiness Recommendation:\n")
  cat("The number of deliveries is expected to increase.\n")
  cat("The company should hire additional delivery partners.\n")
} else{
  cat("\nBusiness Recommendation:\n")
  cat("Current number of delivery partners is sufficient.\n")
}
```

Output

Trend Equation: Y = 7600 + 380 * X
Predicted Deliveries for July = 10260
Business Recommendation:
The number of deliveries is expected to increase.
The company should hire additional delivery partners.



Practical 7:

Seasonal Variation – Simple Method in R (with Realistic Example)

Problem Statement

A cold drink company wants to study its **quarterly sales** over two years to understand the seasonal pattern. The management believes that sales increase during the summer season.

The sales data (in thousands of bottles) is given below:

Quarter	Year 1	Year 2
Q1 (Jan–Mar)	120	130
Q2 (Apr–Jun)	180	195
Q3 (Jul–Sep)	160	170
Q4 (Oct–Dec)	140	150

R Program

```
#----------------------------------------
# Seasonal Variation - Simple Average Method
#----------------------------------------

# Step 1: Enter quarterly sales data
sales <- c(120,180,160,140,
           130,195,170,150)

# Display original data
print("Quarterly Sales Data")
print(sales)

# Step 2: Calculate Overall Average
overall_average <- mean(sales)

cat("\nOverall Average =", overall_average,"\n")

# Step 3: Arrange data into matrix
sales_matrix <- matrix(sales,
                       nrow = 2,
                       byrow = TRUE)
```

```
colnames(sales_matrix) <- c("Q1","Q2","Q3","Q4")
rownames(sales_matrix) <- c("Year1","Year2")

print("\nSales Matrix")
print(sales_matrix)

# Step 4: Calculate Seasonal Average
seasonal_average <- colMeans(sales_matrix)

cat("\nSeasonal Average\n")
print(seasonal_average)

# Step 5: Calculate Seasonal Index
seasonal_index <- (seasonal_average / overall_average) * 100

cat("\nSeasonal Index\n")
print(round(seasonal_index,2))

# Step 6: Interpretation
cat("\nInterpretation:\n")

for(i in 1:4)
{
  if(seasonal_index[i] > 100)
  {
    cat(colnames(sales_matrix)[i],
        "- Sales are ABOVE average.\n")
  }
  else
  {
    cat(colnames(sales_matrix)[i],
        "- Sales are BELOW average.\n")
  }
}
```

Output

Quarterly Sales Data

120 180 160 140 130 195 170 150

Overall Average = 155.625

Sales Matrix

```
       Q1  Q2  Q3  Q4
Year1  120 180 160 140
Year2  130 195 170 150
```

Seasonal Average

```
   Q1    Q2    Q3    Q4
125.0 187.5 165.0 145.0
```

Seasonal Index

```
 Q1      Q2      Q3      Q4
80.33 120.48 106.03 93.18
```

Interpretation:
Q1 - Sales are BELOW average.
Q2 - Sales are ABOVE average.
Q3 - Sales are ABOVE average.
Q4 - Sales are BELOW average.

Real-Life Applications

1. **Ice Cream Company** – Sales peak during summer.
2. **Umbrella Store** – Highest sales during the rainy season.
3. **Air Conditioner Company** – Maximum demand in summer.
4. **Travel Agency** – More bookings during vacation seasons.
5. **E-commerce Website** – Higher sales during festive seasons such as Diwali or Christmas.

Seasonal Variation – Simple Method (R Program Line-by-Line Explanation)

This program calculates the **Seasonal Index** using the **Simple Average Method**. Below is the explanation of every line of code.

Step 1: Enter Quarterly Sales Data

```
sales <- c(120,180,160,140,
           130,195,170,150)
```

Explanation

- `sales` is a variable that stores the quarterly sales data.
- `<-` is the **assignment operator** in R. It stores the value on the right into the variable on the left.
- `c()` stands for **combine**. It combines multiple values into a single vector.
- Here, eight sales values are stored.

Stored Data

Year	Quarter	Sales
1	Q1	120
1	Q2	180
1	Q3	160
1	Q4	140

2	Q1	130
2	Q2	195
2	Q3	170
2	Q4	150

Display Original Data

print("Quarterly Sales Data")

Explanation

- `print()` displays text or values on the console.
- This line prints the heading **Quarterly Sales Data**.

Output

[1] "Quarterly Sales Data"

---

print(sales)

Explanation

- Prints all values stored in the `sales` vector.

Output

[1] 120 180 160 140 130 195 170 150

Step 2: Calculate Overall Average

overall_average <- mean(sales)

Explanation

- `mean()` calculates the arithmetic average.
- It adds all sales values and divides by the total number of observations.
- The result is stored in the variable `overall_average`.

Formula

[
\text{Mean}=\frac{\text{Sum of all values}}{\text{Number of values}}
]

Calculation

120+180+160+140+130+195+170+150 = 1245

1245 ÷ 8 = 155.625

So,

overall_average = 155.625

---

cat("\nOverall Average =", overall_average,"\n")

Explanation

- `cat()` stands for **concatenate and print**.
- It prints text and variables together.
- `\n` means **new line**.
- `overall_average` prints the calculated average.

Output

Overall Average = 155.625

Step 3: Arrange Data into Matrix

```
sales_matrix <- matrix(sales,
                       nrow = 2,
                       byrow = TRUE)
```

Explanation

- `matrix()` converts the vector into a two-dimensional table.
- `sales` is the source data.
- `nrow = 2` creates two rows.
- `byrow = TRUE` fills the matrix row by row.
- The matrix is stored in `sales_matrix`.

Matrix Created

Year1	120	180	160	140
Year2	130	195	170	150

Without `byrow = TRUE`, R would fill the matrix column-wise.

```
colnames(sales_matrix) <- c("Q1","Q2","Q3","Q4")
```

Explanation

- `colnames()` assigns names to the columns.
- `c()` creates a vector of column names.
- The columns become:

Q1

Q2

Q3

Q4

---

rownames(sales_matrix) <- c("Year1","Year2")

Explanation

- `rownames()` assigns names to the rows.
- Row names become:

Year1

Year2

---

print("\nSales Matrix")

Explanation

Prints the heading.

---

print(sales_matrix)

Explanation

Displays the complete matrix.

Output

Q1 Q2 Q3 Q4

Year1 120 180 160 140

Year2 130 195 170 150

Step 4: Calculate Seasonal Average

seasonal_average <- colMeans(sales_matrix)

Explanation

- `colMeans()` calculates the average of each column.
- Since every column represents one quarter, it finds the average sales for each quarter over two years.
- Results are stored in `seasonal_average`.

Calculation

Q1

(120+130)/2 = 125

Q2

(180+195)/2 = 187.5

Q3

(160+170)/2 = 165

Q4

(140+150)/2 = 145

---

cat("\nSeasonal Average\n")

Explanation

Prints the heading.

---

print(seasonal_average)

Explanation

Displays the quarterly averages.

Output

Q1 Q2 Q3 Q4

125 187.5 165 145

Step 5: Calculate Seasonal Index

seasonal_index <- (seasonal_average / overall_average) * 100

Explanation

This calculates the **Seasonal Index**.

Formula

$$
\text{Seasonal Index}=
\frac{\text{Seasonal Average}}
{\text{Overall Average}}
\times100
$$

For Q2

187.5 / 155.625 × 100

=120.48

The result is stored in `seasonal_index`.

---

cat("\nSeasonal Index\n")

Explanation

Prints the heading.

---

print(round(seasonal_index,2))

Explanation

- `round()` rounds decimal values.
- `2` means keep **2 decimal places**.

Without rounding

120.4819277

After rounding

120.48

Output

Q1 80.33

Q2 120.48

Q3 106.03

Q4 93.18

Step 6: Interpretation

cat("\nInterpretation:\n")

Explanation

Prints the heading before the interpretation.

---

for(i in 1:4)

Explanation

- `for` is a loop.
- `i` is the loop variable.
- `1:4` means the loop runs four times:
    - 1 = Q1
    - 2 = Q2
    - 3 = Q3
    - 4 = Q4

{

Explanation

Marks the beginning of the loop body.

---

if(seasonal_index[i] > 100)

Explanation

- `if` checks a condition.
- `seasonal_index[i]` gets the seasonal index for the current quarter.
- If it is greater than 100, sales are above average.

{

Begins the `if` block.

---

cat(colnames(sales_matrix)[i],

Explanation

- `colnames(sales_matrix)` returns:

Q1 Q2 Q3 Q4

- `[i]` selects the current quarter.

For example:

When `i = 2`

Q2

"- Sales are ABOVE average.\n")

Explanation

Prints a message such as:

Q2 - Sales are ABOVE average.

}

Ends the `if` block.

---

else

Explanation

If the seasonal index is **100 or less**, this block executes.

{

Begins the `else` block.

---

cat(colnames(sales_matrix)[i],

Prints the quarter name.

"- Sales are BELOW average.\n")

Prints the message:

Q1 - Sales are BELOW average.

}

Ends the `else` block.

}

Ends the `for` loop.

Final Output

Quarterly Sales Data

120 180 160 140 130 195 170 150

Overall Average = 155.625

Sales Matrix

	Q1	Q2	Q3	Q4
Year1	120	180	160	140
Year2	130	195	170	150

Seasonal Average

Q1	Q2	Q3	Q4
125	187.5	165	145

Seasonal Index

Q1	80.33
Q2	120.48
Q3	106.03
Q4	93.18

Interpretation

Q1 - Sales are BELOW average.

Q2 - Sales are ABOVE average.

Q3 - Sales are ABOVE average.

Q4 - Sales are BELOW average.

Key R Functions Used

Function	Purpose	Example
`c()`	Combines values into a vector	`c(10,20,30)`
`<-`	Assigns a value to a variable	`x <- 5`
`print()`	Displays output	`print(x)`
`cat()`	Prints text and variables together	`cat("Total =", x)`
`mean()`	Calculates the average	`mean(c(2,4,6))`
`matrix()`	Creates a matrix	`matrix(1:6, nrow=2)`
`colnames()`	Assigns or retrieves column names	`colnames(m) <- c("A","B")`
`rownames()`	Assigns or retrieves row names	`rownames(m) <- c("R1","R2")`
`colMeans()`	Calculates the mean of each column	`colMeans(m)`
`round()`	Rounds numbers to a specified number of decimal places	`round(3.14159, 2)`
`for`	Repeats a block of code	`for(i in 1:3)`
`if...else`	Performs conditional execution	`if(x > 10) ... else ...`

Practical 8

B.Sc. Data Science Practical Assignment (R Programming Only)

Title: Sales Forecasting and Time Series Analysis Using R Programming

Subject: Forecasting Techniques using R

Maximum Marks: 50

Software Required: R and RStudio

Real-Life Scenario

You have been appointed as a **Junior Data Analyst** at **FreshMart Supermarket**, Mumbai. The management wants to analyze the monthly sales of **Fruit Juice** over the last two years to understand customer buying behavior, identify trends and seasonal patterns, and forecast future sales.

Using **R Programming**, perform all the following tasks.

Dataset

```
Month <- c("Jan-25","Feb-25","Mar-25","Apr-25","May-25","Jun-25",
        "Jul-25","Aug-25","Sep-25","Oct-25","Nov-25","Dec-25",
        "Jan-26","Feb-26","Mar-26","Apr-26","May-26","Jun-26",
        "Jul-26","Aug-26","Sep-26","Oct-26","Nov-26","Dec-26")


Sales <- c(1200,1250,1400,1650,1900,2100,
        1850,1750,1600,1700,1950,2300,
        1300,1350,1500,1750,2000,2250,
```

1950,1850,1700,1800,2100,2450)

Task 1 – Data Collection, Classification and Presentation (8 Marks)

Write an R program to:

1. Create a data frame.
2. Display the dataset.
3. Display structure of dataset.
4. Display summary statistics.
5. Display first five records.
6. Display last five records.

Expected R Functions

- `data.frame()`
- `str()`
- `summary()`
- `head()`
- `tail()`

Task 2 – Descriptive Statistics (7 Marks)

Write an R program to calculate:

- Total Sales
- Average Sales
- Median Sales
- Maximum Sales
- Minimum Sales
- Standard Deviation
- Variance
- Range

Functions

- `sum()`
- `mean()`

- `median()`
- `max()`
- `min()`
- `sd()`
- `var()`
- `range()`

Task 3 – Data Visualization (8 Marks)

Write R programs to create:

1. Bar Chart
2. Line Chart
3. Pie Chart
4. Histogram
5. Box Plot

Add appropriate:

- Title
- X-axis Label
- Y-axis Label
- Colors

Task 4 – Components of Time Series (6 Marks)

Using R,

1. Convert sales data into a Time Series object.

ts()

2. Plot the time series.
3. Identify:

- Trend
- Seasonal Pattern
- Cyclical Pattern
- Irregular Variation

Write a short interpretation for each.

Task 5 – Moving Average (Simple and Weighted) (8 Marks)

(A) Simple Moving Average

Calculate

- 3-Month Moving Average
- 4-Month Moving Average

Display the result.

(B) Weighted Moving Average

Use the following weights:

Month	Weight
Oldest	0.2
Middle	0.3
Latest	0.5

Calculate the weighted moving average and forecast the next month's sales.

Task 6 – Trend Analysis (Linear Trend Method) (7 Marks)

Using R,

1. Create a time variable.
2. Fit the regression model.

lm()

3. Display the regression equation.
4. Predict sales for January 2027.
5. Plot

- Actual Sales
- Trend Line

Task 7 – Seasonal Variation (Simple Method) (6 Marks)

Using R,

1. Arrange data quarter-wise.
2. Calculate

- Overall Average
- Seasonal Average
- Seasonal Index

3. Display the seasonal index.
4. Identify the quarter having maximum demand.

Task 8 – Business Interpretation (Optional Bonus – 5 Marks)

Answer the following based on the R output:

1. Which month recorded the highest sales?
2. Which month recorded the lowest sales?
3. Is the sales trend increasing or decreasing?
4. Which quarter has the highest seasonal index?
5. Forecast the expected sales for the next month.
6. Suggest three business decisions based on your analysis.

Expected R Concepts Covered

Students should use the following R functions:

Topic	R Functions
Data Frame	`data.frame()`
Data Display	`print()`, `head()`, `tail()`
Dataset Information	`str()`, `summary()`
Statistics	`sum()`, `mean()`, `median()`, `sd()`, `var()`
Graphs	`barplot()`, `plot()`, `pie()`, `hist()`, `boxplot()`
Time Series	`ts()`, `plot()`
Moving Average	`filter()` or manual calculation
Trend Analysis	`lm()`, `predict()`
Seasonal Analysis	`matrix()`, `colMeans()`, `mean()`
Decision Making	`if`, `else`, `for`

Submission Guidelines

Students must submit:

1. **R Script (.R)** containing all programs with comments.
2. **Output Screenshots** of all programs.

3. **A 3–5 page report** containing:
    - Objective
    - Dataset
    - R Code
    - Output
    - Graphs
    - Interpretation
    - Conclusion

Learning Outcomes

After completing this assignment, students will be able to:

1. Create and manipulate datasets in R.
2. Perform descriptive statistical analysis.
3. Visualize business data using charts.
4. Analyze time series data.
5. Apply simple and weighted moving average forecasting.
6. Perform linear trend analysis using regression.
7. Calculate seasonal indices using the simple method.
8. Interpret forecasting results and recommend business decisions based on data.

This assignment covers **all the topics in your syllabus** while giving students practical experience similar to the work performed by a **Data Analyst in a retail organization**.

Module 2

Concept of Correlation

Correlation is a statistical technique used to measure the relationship between two variables. It tells us whether the variables increase or decrease together.

genui{"inference_regression_ml_learning_block":{"type_id":"CORRELATION"}}Types of Correlation

1. Positive Correlation (r > 0):

As one variable increases, the other also increases.

Example: Hours studied and exam marks.

2. Negative Correlation (r < 0):

As one variable increases, the other decreases.

Example: Speed of a vehicle and time taken to reach a destination.

3. Zero Correlation (r = 0):

There is no relationship between the variables.

Example: Shoe size and intelligence.

Correlation Coefficient Formula (Pearson's r)

$$r=\frac{n\sum xy-\sum x\sum y}{\sqrt{\left(n\sum x^2-(\sum x)^2\right)\left(n\sum y^2-(\sum y)^2\right)}}$$

Where:

= First variable

= Second variable

= Number of observations

Example

Student	Hours Studied (x)	Marks (y)
A	2	40
B	4	50
C	6	65
D	8	80

E 10 90

Calculate:

Substituting into the formula:

$$r=\frac{5(2180)-30(325)}{\sqrt{(5\times220-30^2)(5\times22725-325^2)}}$$

$$=\frac{1150}{1160.6}$$

$$\approx 0.99$$

Interpretation

r = 0.99 indicates a very strong positive correlation.

This means students who study more hours tend to score higher marks.

Real-Life Examples

Positive: Advertising expenditure ↔ Sales.

Negative: Product price ↔ Customer demand.

Zero: Mobile phone number ↔ Examination marks.

Module 2: Regression and Business Forecasting Applications (30 hrs)

1. Concept of correlation

2. Karl Pearson correlation coefficient

Solution:

```
# ============================================================

# CONCEPT OF CORRELATION

# Pearson's Correlation Coefficient using R

# ============================================================


# Clear the console

cat("\014")


# ------------------------------------------------------

# Step 1: Enter the data

# ------------------------------------------------------


study_hours <- c(1, 2, 3, 4, 5)

marks <- c(35, 40, 50, 55, 65)


# Number of observations

n <- length(study_hours)


# ------------------------------------------------------

# Step 2: Display the original data

# ------------------------------------------------------


cat("==========================================\n")
```

```
cat("       CORRELATION ANALYSIS IN R\n")

cat("=========================================\n\n")


cat("Original Data:\n")


data <- data.frame(

  Student = c("A", "B", "C", "D", "E"),

  Study_Hours = study_hours,

  Marks = marks

)


print(data)


# ------------------------------------------------------

# Step 3: Prepare the calculation table

# ------------------------------------------------------


X <- study_hours

Y <- marks


X2 <- X^2

Y2 <- Y^2

XY <- X * Y


calculation_table <- data.frame(

  X = X,

  Y = Y,

  X2 = X2,
```

```
  Y2 = Y2,
  XY = XY
)

cat("\nCalculation Table:\n")
print(calculation_table)

# ------------------------------------------------------
# Step 4: Calculate the totals
# ------------------------------------------------------

sum_X <- sum(X)
sum_Y <- sum(Y)
sum_X2 <- sum(X2)
sum_Y2 <- sum(Y2)
sum_XY <- sum(XY)

cat("\n==========================================\n")
cat("            SUMMATION VALUES\n")
cat("==========================================\n")

cat("n       =", n, "\n")
cat("Sum X   =", sum_X, "\n")
cat("Sum Y   =", sum_Y, "\n")
cat("Sum X2  =", sum_X2, "\n")
cat("Sum Y2  =", sum_Y2, "\n")
cat("Sum XY  =", sum_XY, "\n")
```

```
# ----------------------------------------------------

# Step 5: Calculate Pearson's correlation manually

# ----------------------------------------------------


numerator <- n * sum_XY - (sum_X * sum_Y)


denominator <- sqrt(

  (n * sum_X2 - sum_X^2) *

    (n * sum_Y2 - sum_Y^2)

)


r <- numerator / denominator


# ----------------------------------------------------

# Step 6: Display calculation

# ----------------------------------------------------


cat("\n============================================\n")

cat("       PEARSON CORRELATION CALCULATION\n")

cat("============================================\n")


cat("\nNumerator:\n")

cat("n(Sum XY) - (Sum X)(Sum Y)\n")

cat(n, "*", sum_XY, "-", sum_X, "*", sum_Y, "\n")

cat("=", numerator, "\n")


cat("\nDenominator:\n")

cat("sqrt[(n(Sum X2)-(Sum X)^2) *\n")
```

```
cat("     (n(Sum Y2)-(Sum Y)^2)]\n")

cat("=", denominator, "\n")


cat("\nPearson Correlation Coefficient:\n")

cat("r =", round(r, 3), "\n")


# ------------------------------------------------------

# Step 7: Verify using R's built-in cor() function

# ------------------------------------------------------


r_builtin <- cor(X, Y, method = "pearson")


cat("\nUsing R built-in cor() function:\n")

cat("r =", round(r_builtin, 3), "\n")


# ------------------------------------------------------

# Step 8: Interpret the result

# ------------------------------------------------------


cat("\n============================================\n")

cat("              INTERPRETATION\n")

cat("============================================\n")


if (r > 0.90) {

  cat("There is a VERY STRONG POSITIVE correlation.\n")

} else if (r > 0.70) {

  cat("There is a STRONG POSITIVE correlation.\n")

} else if (r > 0.40) {
```

```
  cat("There is a MODERATE POSITIVE correlation.\n")
} else if (r > 0.10) {
  cat("There is a WEAK POSITIVE correlation.\n")
} else if (r < -0.90) {
  cat("There is a VERY STRONG NEGATIVE correlation.\n")
} else if (r < -0.70) {
  cat("There is a STRONG NEGATIVE correlation.\n")
} else if (r < -0.40) {
  cat("There is a MODERATE NEGATIVE correlation.\n")
} else if (r < -0.10) {
  cat("There is a WEAK NEGATIVE correlation.\n")
} else {
  cat("There is NO significant linear correlation.\n")
}

cat("\nConclusion:\n")
cat("As study hours increase, examination marks also tend\n")
cat("to increase. Therefore, the data shows a very strong\n")
cat("positive linear relationship between study hours and marks.\n")

# ------------------------------------------------------
# Step 9: Scatter Plot
# ------------------------------------------------------

plot(
  X, Y,
  main = "Correlation between Study Hours and Marks",
  xlab = "Study Hours",
```

```
  ylab = "Marks",
  pch = 19
)

# Add regression line
abline(lm(Y ~ X), lwd = 2)

# Display correlation value on the plot
legend(
  "topleft",
  legend = paste("Pearson r =", round(r, 3)),
  bty = "n"
)
```

Output

```
==========================================
      CORRELATION ANALYSIS IN R
==========================================

Original Data:
  Student Study_Hours Marks
1       A           1    35
2       B           2    40
3       C           3    50
4       D           4    55
5       E           5    65

Calculation Table:
  X  Y X2   Y2  XY
```

1 1 35 1 1225 35

2 2 40 4 1600 80

3 3 50 9 2500 150

4 4 55 16 3025 220

5 5 65 25 4225 325

==========

SUMMATION VALUES

==========

n = 5

Sum X = 15

Sum Y = 245

Sum X2 = 55

Sum Y2 = 12575

Sum XY = 810

==========

PEARSON CORRELATION CALCULATION

==========

Numerator:

n(Sum XY) - (Sum X)(Sum Y)

5 * 810 - 15 * 245

= 375

Denominator:

sqrt[(n(Sum X2)-(Sum X)^2) *

(n(Sum Y2)-(Sum Y)^2)]

= 377.492

Pearson Correlation Coefficient:

r = 0.993

Using R built-in cor() function:

r = 0.993

==========

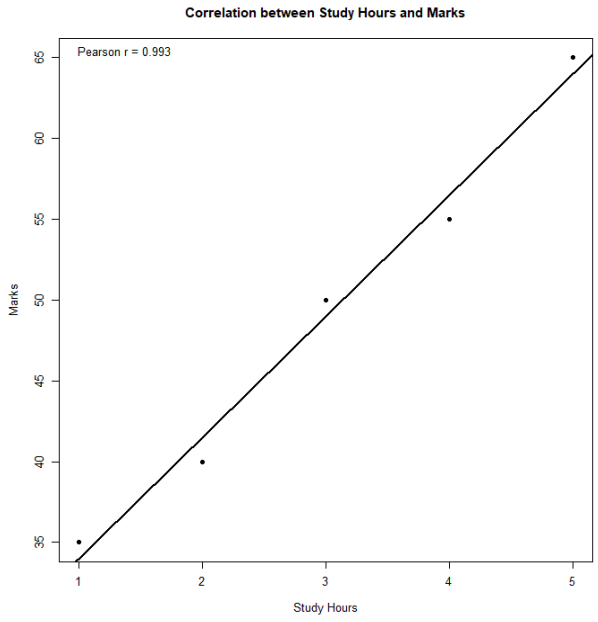
INTERPRETATION

==========

There is a VERY STRONG POSITIVE correlation.

Conclusion:

As study hours increase, examination marks also tend to increase. Therefore, the data shows a very strong positive linear relationship between study hours and marks.



3. Linear regression method

Program:

```
# ----------------------------------------
# LINEAR REGRESSION - E-COMMERCE CASE STUDY
# ----------------------------------------

# Step 1: Create the dataset

advertising <- c(10, 12, 15, 18, 20, 22, 25, 28, 30, 32)

sales <- c(50, 55, 60, 65, 70, 75, 80, 85, 90, 95)

# Create data frame
ecommerce <- data.frame(advertising, sales)

# Display data
print(ecommerce)


# Step 2: Create Linear Regression Model
# sales is dependent variable
# advertising is independent variable

model <- lm(sales ~ advertising, data = ecommerce)


# Step 3: Display regression model

print(model)


# Step 4: Display detailed summary

summary(model)


# Step 5: Get regression coefficients

coefficients(model)


# Step 6: Predict sales when advertising
# expenditure is ₹35,000

new_data <- data.frame(advertising = 35)

predicted_sales <- predict(model, newdata = new_data)
```

```
print(predicted_sales)


# Step 7: Display prediction in a meaningful format

cat("Advertising Expenditure: ₹35,000\n")
cat("Predicted Sales Revenue: ₹",
    round(predicted_sales, 2), " thousand\n")


# Step 8: Plot the original data

plot(
  ecommerce$advertising,
  ecommerce$sales,
  main = "Advertising Expenditure vs Sales",
  xlab = "Advertising Expenditure (₹000)",
  ylab = "Sales Revenue (₹000)",
  pch = 19
)


# Step 9: Add regression line

abline(model, lwd = 2)


# Step 10: Predict sales for multiple advertising budgets

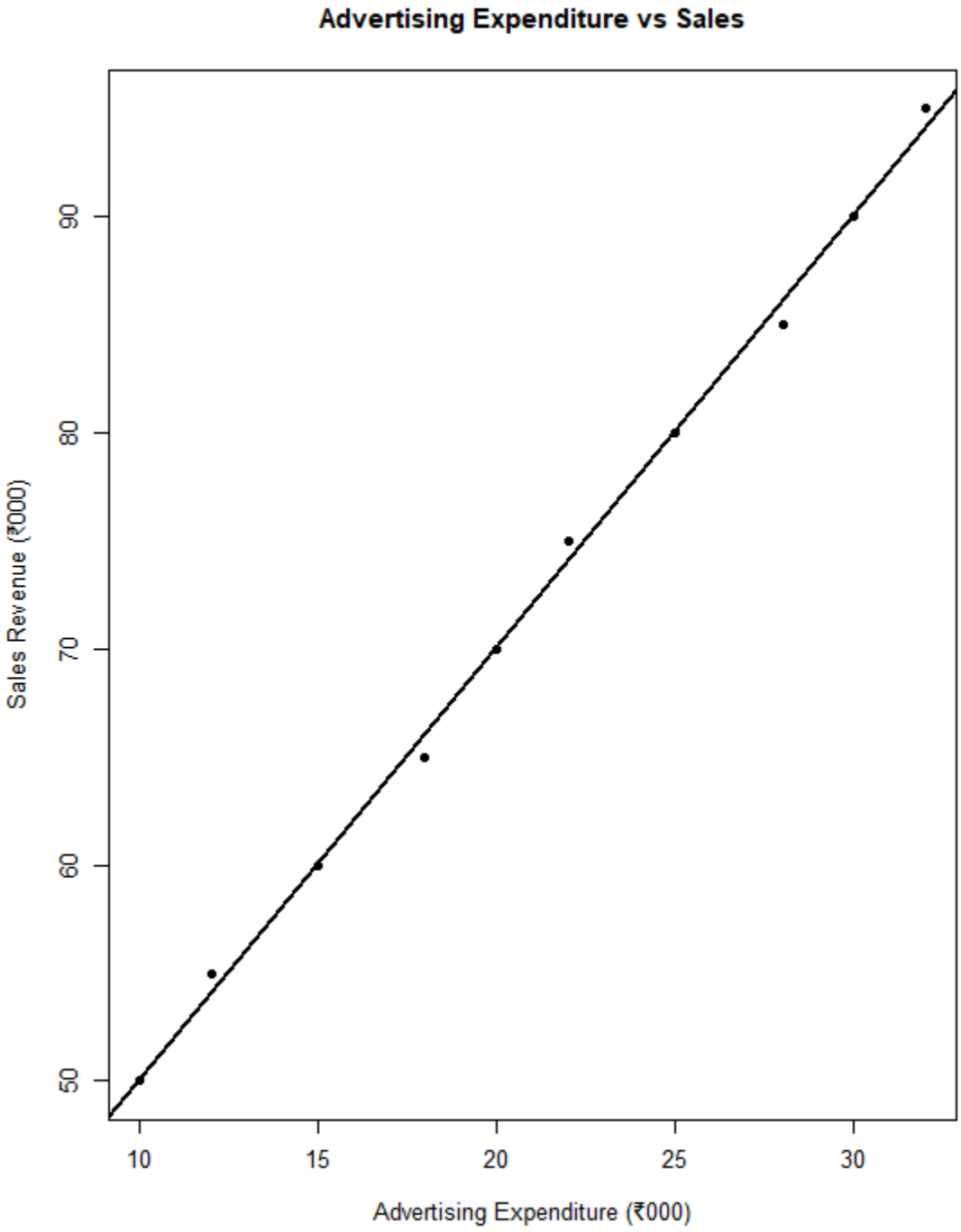
future_advertising <- data.frame(
  advertising = c(15, 20, 25, 30, 35, 40)
)

future_predictions <- predict(
  model,
  newdata = future_advertising
)

prediction_table <- data.frame(
  Advertising = future_advertising$advertising,
  Predicted_Sales = round(future_predictions, 2)
)

print(prediction_table)
```

Output


Advertising Expenditure vs Sales
Sales Revenue (₹000)
50
60
70
80
90
10
15
20
25
30
Advertising Expenditure (₹000)

Explanation of code

Line-by-Line Explanation of the R Linear Regression Code

This program solves an e-commerce sales prediction problem using Simple Linear Regression.

Business situation

The company wants to answer:

> "If we spend ₹35,000 on advertising, how much sales revenue can we expect?"

Here:

Advertising = Independent Variable (X)

Sales = Dependent Variable (Y)

---

1. Program Heading

```
# ------------------------------------------
# LINEAR REGRESSION - E-COMMERCE CASE STUDY
# ------------------------------------------
```

These lines start with #.

In R, anything after # on that line is a comment.

Comments are not executed by R. They are written to make the program easier to understand.

So:

LINEAR REGRESSION - E-COMMERCE CASE STUDY

means that the program is about applying linear regression to an e-commerce problem.

Step 1: Create the Dataset

2. Advertising data

```
advertising <- c(10, 12, 15, 18, 20, 22, 25, 28, 30, 32)
```

This creates a variable called advertising.

c()

The c() function means combine.

It combines multiple values into a vector.

So:

c(10, 12, 15, 18, 20, 22, 25, 28, 30, 32)

creates:

10 12 15 18 20 22 25 28 30 32

The values represent advertising expenditure in ₹ thousands.

For example:

10 = ₹10,000
20 = ₹20,000
32 = ₹32,000

---

3. Sales data

sales <- c(50, 55, 60, 65, 70, 75, 80, 85, 90, 95)

This creates another variable called sales.

Again, c() combines the values into a vector.

These values represent sales revenue in ₹ thousands.

For example:

50 = ₹50,000
70 = ₹70,000
95 = ₹95,000

The two vectors have the same number of observations.

Advertising	Sales
10	50
12	55

15 60
18 65
20 70
22 75
25 80
28 85
30 90
32 95

Step 2: Create a Data Frame

4. Create the e-commerce dataset

ecommerce <- data.frame(advertising, sales)

data.frame() creates a data frame.

A data frame is similar to a table in Excel.

The result will look like:

advertising sales
1 10 50
2 12 55
3 15 60
4 18 65
5 20 70
6 22 75
7 25 80
8 28 85
9 30 90
10 32 95

The name of this data frame is:

ecommerce

---

5. Display the dataset

print(ecommerce)

print() displays an object in the R console.

Here it displays the contents of the ecommerce data frame.

Output:

```
advertising sales
1          10    50
2          12    55
3          15    60
...
```

Step 3: Create the Linear Regression Model

6. Create the model

```
model <- lm(sales ~ advertising, data = ecommerce)
```

This is the most important line in the program.

Let's break it down.

lm()

lm() means:

> Linear Model

It is the R function used to create a linear regression model.

---

sales

sales

is the dependent variable (Y).

We want to predict sales.

~

The tilde symbol ~ means:

> "is explained by" or "depends on"

Therefore:

sales ~ advertising

means:

> Sales depends on advertising expenditure.

Mathematically:

$Y = a + bX$

where:

Y = Sales

X = Advertising

a = Intercept

b = Slope

---

advertising

advertising

is the independent variable (X).

It is used to predict sales.

---

data = ecommerce

data = ecommerce

tells R:

> Find the sales and advertising variables inside the ecommerce data frame.

---

model <-

The result of the regression is stored in:

model

So:

model <- lm(...)

means:

> Create a linear regression model and store it in the variable model.

Step 4: Display the Regression Model

7. Print the model

print(model)

This displays the regression equation estimated by R.

You will get something similar to:

Call:
lm(formula = sales ~ advertising, data = ecommerce)

Coefficients:
(Intercept) advertising
    ...
    ...

The two important coefficients are:

1. Intercept

2. Advertising coefficient

These coefficients are used to construct the regression equation.

Step 5: Display Detailed Model Information

8. Summary of the model

summary(model)

This gives detailed information about the regression model.

It typically displays:

Call:
Coefficients:
Residuals:
Multiple R-squared:
Adjusted R-squared:
F-statistic:
p-value:

Important information

Coefficients

Shows:

(Intercept)
advertising

These are used to create the regression equation.

R-squared

R-squared tells us how much of the variation in sales is explained by advertising in the fitted linear model.

For example, an R^2 close to 1 indicates a very strong linear fit for this dataset.

p-value

The p-value associated with the advertising coefficient helps assess whether the relationship is statistically significant under the model assumptions.

Step 6: Get Regression Coefficients

9. Extract coefficients

coefficients(model)

The coefficients() function extracts the regression coefficients from the model.

It returns values such as:

```
(Intercept) advertising
      a           b
```

Suppose the output were:

```
(Intercept) advertising
  33.78       1.91
```

Then the regression equation would be approximately:

Sales = 33.78 + 1.91(Advertising)

The actual values displayed by your R installation should be used.

Step 7: Prepare Data for Prediction

10. Create new advertising value

new_data <- data.frame(advertising = 35)

We want to predict sales when advertising expenditure is:

35

Since advertising is measured in ₹ thousands:

35 = ₹35,000

We create a data frame because predict() expects new observations in the appropriate variable structure.

The resulting data frame is:

```
advertising
1          35
```

Step 8: Predict Sales

11. Use the model for prediction

predicted_sales <- predict(model, newdata = new_data)

This is another important line.

predict()

predict() uses the regression model to calculate a predicted value.

model

model

tells R which regression model to use.

newdata = new_data

newdata = new_data

tells R:

> Use the new advertising value of 35 and calculate the corresponding predicted sales.

The result is stored in:

predicted_sales

Step 9: Display the Prediction

12. Print the predicted value

print(predicted_sales)

This displays the predicted sales value in the R console.

For example:

100.6

Because sales are measured in ₹ thousands:

100.6 thousand = ₹100,600

Step 10: Display a User-Friendly Message

13. Display advertising expenditure

```
cat("Advertising Expenditure: ₹35,000\n")
```

cat() is used to display text and values.

The text displayed is:

Advertising Expenditure: ₹35,000

What is \n?

\n means:

> Move to a new line.

---

14. Display predicted sales

```
cat("Predicted Sales Revenue: ₹",
    round(predicted_sales, 2), " thousand\n")
```

This line displays the predicted sales in a readable format.

round()

round(predicted_sales, 2)

rounds the prediction to 2 decimal places.

For example:

100.59678

becomes:

100.60

Complete output

It may look like:

Advertising Expenditure: ₹35,000
Predicted Sales Revenue: ₹ 100.60 thousand

Step 11: Create the Scatter Plot

15. Start plotting

plot(

plot() is used to create a graph.

Here we want to display the relationship between:

Advertising expenditure

Sales revenue

---

16. X-axis data

ecommerce$advertising,

The $ operator is used to access a particular column from a data frame.

Therefore:

ecommerce$advertising

means:

> Get the advertising column from the ecommerce data frame.

This becomes the X-axis.

---

17. Y-axis data

ecommerce$sales,

This means:

> Get the sales column from the ecommerce data frame.

This becomes the Y-axis.

Therefore:

plot(ecommerce$advertising, ecommerce$sales)

means:

> Plot advertising against sales.

---

18. Graph title

main = "Advertising Expenditure vs Sales",

main specifies the main title of the graph.

The graph title will be:

Advertising Expenditure vs Sales

---

19. X-axis label

xlab = "Advertising Expenditure (₹000)",

xlab means X-axis label.

It tells the user that the X-axis represents advertising expenditure in thousands of rupees.

---

20. Y-axis label

ylab = "Sales Revenue (₹000)",

ylab means Y-axis label.

It tells the user that sales revenue is measured in thousands of rupees.

---

21. Point style

pch = 19

pch means plotting character.

pch = 19 displays the observations as filled circular points.

So the graph shows the actual e-commerce observations as dots.

---

22. Close the plot function

)

This closes the plot() function.

Step 12: Add the Regression Line

23. Add regression line

abline(model, lwd = 2)

abline() adds a straight line to an existing graph.

Here:

abline(model)

takes the intercept and slope from the regression model and draws the best-fit regression line.

lwd = 2

lwd means line width.

lwd = 2

makes the regression line thicker.

The graph now contains:

Actual observations → dots

Regression model → straight line

Step 13: Predict Multiple Advertising Budgets

24. Create future advertising values

```
future_advertising <- data.frame(
  advertising = c(15, 20, 25, 30, 35, 40)
)
```

Instead of predicting sales for only ₹35,000, we now want predictions for several advertising budgets.

The values are:

15
20
25
30
35
40

Because the unit is ₹000:

₹15,000
₹20,000
₹25,000
₹30,000
₹35,000
₹40,000

The result is a data frame:

```
advertising
1          15
2          20
3          25
```

4 30
5 35
6 40

Step 14: Predict All Values

25. Generate multiple predictions

```
future_predictions <- predict(
  model,
  newdata = future_advertising
)
```

This uses the same regression model to predict sales for all six advertising budgets.

R performs:

```
Advertising → Predicted Sales
15          → Prediction
20          → Prediction
25          → Prediction
30          → Prediction
35          → Prediction
40          → Prediction
```

The results are stored in:

```
future_predictions
```

Step 15: Create a Prediction Table

26. Create the final table

```
prediction_table <- data.frame(
```

This creates a new data frame called:

```
prediction_table
```

The purpose is to combine advertising expenditure and predicted sales into one table.

---

27. Add advertising values

Advertising = future_advertising$advertising,

This creates a column called:

Advertising

It takes the values from:

future_advertising$advertising

So the column contains:

15
20
25
30
35
40

---

28. Add predicted sales

Predicted_Sales = round(future_predictions, 2)

This creates another column:

Predicted_Sales

round(..., 2) ensures that the predictions are displayed to two decimal places.

The final table will look approximately like:

Advertising (₹000)	Predicted Sales (₹000)
15	62.44
20	71.98
25	81.53
30	91.07
35	100.62
40	110.16

The exact values should be taken from R's output.

Step 16: Display the Prediction Table

29. Print the table

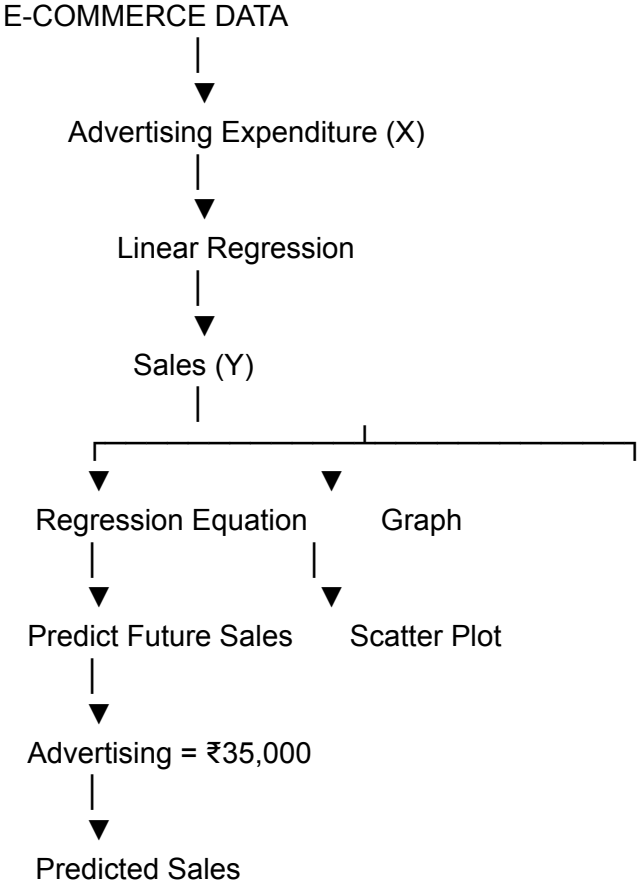
print(prediction_table)

This displays the final prediction table in the R console.

The company can now compare different advertising budgets with their corresponding predicted sales.

Complete Flow of the Program

The entire program follows this sequence:



Most Important R Functions

R Code	Purpose
c()	Creates a vector
data.frame()	Creates a tabular dataset
print()	Displays output

lm()	Creates a linear regression model
summary()	Provides detailed model statistics
coefficients()	Extracts regression coefficients
predict()	Predicts future values
plot()	Creates a graph
abline()	Adds regression line
round()	Rounds numerical values
cat()	Displays formatted text
$	Accesses a data-frame column

In one sentence

The key line is:

model <- lm(sales ~ advertising, data = ecommerce)

It tells R:

> "Build a linear model that predicts e-commerce sales using advertising expenditure."

Then:

predict(model, newdata = new_data)

tells R:

> "Using that model, predict sales for a new advertising expenditure."

5. Forecasting using regression equation

Program:

```
# Historical sales data

month <- c(1, 2, 3, 4, 5, 6)

sales <- c(120, 135, 150, 165, 180, 195)

# Create data frame

data <- data.frame(month, sales)

# Display data

print(data)

# Create linear regression model

model <- lm(sales ~ month, data = data)

# Display regression results

summary(model)

# Display coefficients

coefficients(model)

# Forecast July

july <- data.frame(month = 7)

forecast <- predict(model, newdata = july)

# Display forecast

cat("Forecasted Sales for July =",    forecast, "thousand\n")
```

Output

```
      month sales
1      1   120
2      2   135
3      3   150
4      4   165
```

```
5     5   180
6     6   195
```

```
Forecasted Sales for July = 210 thousand
```

Explanation of code

Sure. This R program uses linear regression to study the relationship between month and sales, then predicts sales for July.

Line-by-line explanation

1. Create the month variable

```
month <- c(1, 2, 3, 4, 5, 6)
```

month is the name of the variable.

<- is the assignment operator in R.

c() means combine or concatenate values.

Six month numbers are stored: 1 to 6.

Here, 1 = January, 2 = February, ..., 6 = June.

---

2. Create the sales variable

sales <- c(120, 135, 150, 165, 180, 195)

Creates a variable called sales.

Stores sales values for the six months.

The values are:

Month 1 → 120

Month 2 → 135

Month 3 → 150

Month 4 → 165

Month 5 → 180

Month 6 → 195

The comment later says the values are in thousands, so 120 means ₹120 thousand, for example.

---

3. Create a data frame

```
data <- data.frame(month, sales)
```

data.frame() creates a table-like structure.

It combines month and sales into two columns.

data becomes:

```
month sales

1   120
2   135
3   150
4   165
5   180
```

6 195

---

4. Display the data

```
print(data)
```

print() displays an object in the R console.

Here, it displays the data data frame.

It allows us to verify that the data has been entered correctly.

---

5. Create the linear regression model

```
model <- lm(sales ~ month, data = data)
```

This is the most important line.

lm() means Linear Model.

It is used to perform linear regression.

sales ~ month means:

sales is the dependent/response variable.

month is the independent/predictor variable.

data = data tells R to use the variables from the data data frame.

The resulting regression model is stored in model.

The model is essentially:

Sales = a + b(Month)

For these data:

Sales = 105 + 15(Month)

So, every additional month increases predicted sales by 15 thousand.

---

6. Display regression results

summary(model)

summary() provides detailed information about the regression model.

It displays:

Regression coefficients

Standard errors

t-values

p-values

Residual information

Adjusted

F-statistic

It helps determine how well the regression model explains sales.

For this perfectly linear dataset, the will be 1, meaning the model explains 100% of the variation in the given sales data.

---

7. Display regression coefficients

coefficients(model)

coefficients() extracts the coefficients from the regression model.

It gives two values:

```
(Intercept)    105
month           15
```

Their meanings are:

Intercept = 105

When month = 0, the model predicts sales of 105 thousand.

Month coefficient = 15

For every increase of 1 month, predicted sales increase by 15 thousand.

Therefore:

Sales = 105 + 15(Month)

---

8. Create July data

july <- data.frame(month = 7)

Creates a new data frame called july.

It contains month = 7.

Since June is month 6, month 7 represents July.

The data looks like:

month

7

---

9. Predict July sales

forecast <- predict(model, newdata = july)

predict() uses the trained regression model to make a prediction.

model is the regression model created earlier.

newdata = july tells R to make the prediction for month 7.

The predicted value is stored in forecast.

Using:

Sales = 105 + 15(7)

Sales = 105 + 105

Sales = 210

Therefore, predicted July sales are 210 thousand.

---

10. Display the forecast

```
cat("Forecasted Sales for July =",
forecast, "thousand\n")
```

cat() is used to concatenate and display text and values.

"Forecasted Sales for July =" is the text displayed before the result.

forecast inserts the predicted value.

"thousand" specifies the unit.

\n means new line.

The output will be approximately:

Forecasted Sales for July = 210 thousand

Overall working of the program

The complete process is:

Historical Sales Data → Data Frame →
Linear Regression → Regression Equation →
July Input → Prediction

The regression equation is:

$$\boxed{Sales = 105 + 15 \times Month}$$

For July:

$$\boxed{Sales = 210\text{ thousand}}$$

So the model forecasts July sales = 210 thousand.

Practical 6 and Practical 7

Demand Forecasting and Sales Forecasting using Seasonal variation

Code

```
#
==========================================
====================
# SALES FORECASTING USING SEASONAL
VARIATIONS
# Ratio-to-Average Method
#
==========================================
====================

# Historical quarterly sales data
# Sales are in ₹ lakh

Q1 <- c(80, 100)
Q2 <- c(120, 150)
Q3 <- c(160, 200)
Q4 <- c(100, 125)

# Create data frame
sales_data <- data.frame(
  Year = c("Year 1", "Year 2"),
```

```
  Q1 = Q1,
  Q2 = Q2,
  Q3 = Q3,
  Q4 = Q4
)

# Display historical data
print(sales_data)


#
==========================================
====================
# STEP 1: Calculate Annual Sales
#
==========================================
====================

sales_data$Annual_Sales <- rowSums(
  sales_data[, c("Q1", "Q2", "Q3", "Q4")]
)

print(sales_data)


#
==========================================
====================
```

```
# STEP 2: Calculate Annual Average
#
==========================================
====================

sales_data$Annual_Average <-
sales_data$Annual_Sales / 4

print(sales_data)


#
==========================================
====================
# STEP 3: Calculate Seasonal Relatives
# Formula:
# Seasonal Relative = Quarterly Sales /
Annual Average × 100
#
==========================================
====================

seasonal_relative <- data.frame(
  Q1 = sales_data$Q1 /
sales_data$Annual_Average * 100,
  Q2 = sales_data$Q2 /
sales_data$Annual_Average * 100,
```

```
  Q3 = sales_data$Q3 /
sales_data$Annual_Average * 100,
  Q4 = sales_data$Q4 /
sales_data$Annual_Average * 100
)

cat("\nSeasonal Relatives:\n")
print(seasonal_relative)


#
==========================================
==================
# STEP 4: Calculate Seasonal Indices
# Average seasonal relative for each
quarter
#
==========================================
==================

seasonal_index <-
colMeans(seasonal_relative)

cat("\nSeasonal Indices:\n")
print(seasonal_index)
```

```
#
==========================================
==========================
# STEP 5: Calculate Year 3 Trend
#
==========================================
==========================

year1_sales <- sales_data$Annual_Sales[1]
year2_sales <- sales_data$Annual_Sales[2]

# Increase in annual sales
annual_increase <- year2_sales -
year1_sales

# Forecast Year 3 annual sales
year3_sales <- year2_sales +
annual_increase

cat("\nYear 1 Sales =", year1_sales,
"lakh\n")
cat("Year 2 Sales =", year2_sales,
"lakh\n")
cat("Annual Increase =", annual_increase,
"lakh\n")
cat("Forecasted Year 3 Sales =",
year3_sales, "lakh\n")
```

```
#
==========================================
==========
# STEP 6: Calculate Average Quarterly
Sales for Year 3
#
==========================================
==========

year3_quarterly_average <- year3_sales /
4

cat(
  "Year 3 Average Quarterly Sales =",
  year3_quarterly_average,
  "lakh\n"
)


#
==========================================
==========
# STEP 7: Forecast Year 3 Quarterly Sales
#
# Forecast = Trend Average × Seasonal
Index / 100
```

```
#
==========================================
====================

forecast_Q1 <- year3_quarterly_average *
  seasonal_index["Q1"] / 100

forecast_Q2 <- year3_quarterly_average *
  seasonal_index["Q2"] / 100

forecast_Q3 <- year3_quarterly_average *
  seasonal_index["Q3"] / 100

forecast_Q4 <- year3_quarterly_average *
  seasonal_index["Q4"] / 100


#
==========================================
====================
# STEP 8: Display Forecast
#
==========================================
====================

forecast <- data.frame(
  Quarter = c("Q1", "Q2", "Q3", "Q4"),
```

```
  Seasonal_Index =
as.numeric(seasonal_index),
  Forecast_Sales = c(
    forecast_Q1,
    forecast_Q2,
    forecast_Q3,
    forecast_Q4
  )
)

cat("\nYear 3 Sales Forecast:\n")
print(forecast)


#
==========================================
==================
# STEP 9: Calculate Total Forecast
#
==========================================
==================

total_forecast <-
sum(forecast$Forecast_Sales)

cat(
  "\nTotal Forecasted Sales for Year 3
=",
```

```
  total_forecast,
  "lakh\n"
)
```

Output

```
Year 1 Sales = 460 lakh
Year 2 Sales = 575 lakh
Annual Increase = 115 lakh
Forecasted Year 3 Sales = 690 lakh

Year 3 Average Quarterly Sales = 172.5
lakh

Seasonal Relatives:
         Q1        Q2        Q3        Q4
1 69.56522 104.3478 139.1304 86.95652
2 69.56522 104.3478 139.1304 86.95652

Seasonal Indices:
       Q1         Q2         Q3         Q4
 69.56522 104.34783 139.13043  86.95652

Year 3 Sales Forecast:
   Quarter Seasonal_Index Forecast_Sales
Q1      Q1       69.56522             120
Q2      Q2      104.34783             180
Q3      Q3      139.13043             240
```

```
Q4      Q4       86.95652              150

Total Forecasted Sales for Year 3 = 690
lakh
```

Code Explanation

Below is a line-by-line explanation of your R program. The program performs sales forecasting using seasonal variations with the Ratio-to-Average Method.

---

1. Program Heading

```
#
==========================================
====================
# SALES FORECASTING USING SEASONAL
VARIATIONS
# Ratio-to-Average Method
#
==========================================
====================
```

indicates a comment in R.

Comments are not executed by R.

They are used to explain the purpose of the program.

The program is forecasting sales.

It uses seasonal variations.

Specifically, it uses the
Ratio-to-Average Method.

---

2. Historical Quarterly Sales

Historical quarterly sales data
Sales are in ₹ lakh

These are comments explaining that the following data represents historical quarterly sales.

The unit of sales is ₹ lakh.

Q1

Q1 <- c(80, 100)

Q1 is the variable name.

<- is R's assignment operator.

c() means combine.

80 represents Q1 sales for Year 1.

100 represents Q1 sales for Year 2.

Therefore:

Q1 = (80,100)

Q2

```
Q2 <- c(120, 150)

Stores Q2 sales:

Year 1 = ₹120 lakh

Year 2 = ₹150 lakh



---

Q3

Q3 <- c(160, 200)

Stores Q3 sales:

Year 1 = ₹160 lakh

Year 2 = ₹200 lakh



---

Q4
```

```
Q4 <- c(100, 125)
```

Stores Q4 sales:

Year 1 = ₹100 lakh

Year 2 = ₹125 lakh

---

3. Create Data Frame

```
sales_data <- data.frame(
  Year = c("Year 1", "Year 2"),
  Q1 = Q1,
  Q2 = Q2,
  Q3 = Q3,
  Q4 = Q4
)
```

This creates a data frame, which is similar to a spreadsheet table.

First line

```
sales_data <- data.frame(
```

sales_data is the name of the data frame.

data.frame() creates a table.

<- stores the table in sales_data.

```
Year column

Year = c("Year 1", "Year 2"),
```

Creates a column called Year.

It contains:

Year 1
Year 2

```
Q1 column

Q1 = Q1,
```

Creates the Q1 column using the previously created Q1 variable.

```
Q2 column
```

Q2 = Q2,

Creates the Q2 column.

Q3 column

Q3 = Q3,

Creates the Q3 column.

Q4 column

Q4 = Q4

Creates the Q4 column.

The resulting table is:

Year Q1 Q2 Q3 Q4

Year 1 80 120 160 100
Year 2 100 150 200 125

---

4. Display Historical Data

```
print(sales_data)
```

print() displays an object in the R console.

sales_data is the object being displayed.

This allows us to verify that the historical data has been entered correctly.

---

5. Step 1 — Calculate Annual Sales

```
sales_data$Annual_Sales <- rowSums(
  sales_data[, c("Q1", "Q2", "Q3", "Q4")]
)
```

This is an important calculation.

We want:

\text{Annual Sales}

=
Q1+Q2+Q3+Q4

---

sales_data$Annual_Sales

sales_data$Annual_Sales

$ is used to access or create a column inside a data frame.

Annual_Sales is the new column.

So this line creates:

Annual_Sales

inside sales_data.

---

rowSums()

rowSums(...)

rowSums() calculates the sum of values row by row.

For Year 1:

80+120+160+100=460

For Year 2:

100+150+200+125=575

Therefore:

Year	Annual Sales
Year 1	460
Year 2	575

Selecting quarterly columns

```
sales_data[, c("Q1", "Q2", "Q3", "Q4")]
```

The comma inside:

data[rows, columns]

means:

before comma → rows

after comma → columns

Here no rows are specified, so R selects all rows.

c("Q1", "Q2", "Q3", "Q4")

specifies the four columns to be added.

---

6. Display Updated Data

print(sales_data)

Displays the data frame after adding Annual_Sales.

It will contain:

Year Q1 Q2 Q3 Q4 Annual Sales

Year 1 80 120 160 100 460
Year 2 100 150 200 125 575

---

7. Step 2 — Calculate Annual Average

```
sales_data$Annual_Average <-
sales_data$Annual_Sales / 4
```

This creates another column called:

Annual_Average

The formula is:

$$\text{Annual Average} = \frac{\text{Annual Sales}}{4}$$

Why divide by 4?

Because there are four quarters in a year.

Year 1

\frac{460}{4}=115

Year 2

\frac{575}{4}=143.75

Therefore:

Year Annual Sales Annual Average

Year 1 460 115
Year 2 575 143.75

---

8. Display Annual Average

print(sales_data)

Displays the updated data frame.

---

9. Step 3 — Calculate Seasonal Relatives

seasonal_relative <- data.frame(

Creates a new data frame called seasonal_relative.

This table will contain the seasonal relatives for Q1, Q2, Q3 and Q4.

Q1 Seasonal Relative

Q1 = sales_data$Q1 / sales_data$Annual_Average * 100,

The mathematical formula is:

$$\text{Seasonal Relative} = \frac{\text{Quarterly Sales}}{\text{Annual Average}} \times 100$$

For Year 1:

$$\frac{80}{115}\times100$$
=
69.57

For Year 2:

$$\frac{100}{143.75}\times100$$
=
69.57

Q2 Seasonal Relative

```
Q2 = sales_data$Q2 /
sales_data$Annual_Average * 100,
```

For Year 1:

$$\frac{120}{115}\times100$$
=
104.35

For Year 2:

$$\frac{150}{143.75}\times100$$
=
104.35

Q3 Seasonal Relative

```
Q3 = sales_data$Q3 /
sales_data$Annual_Average * 100,
```

For Year 1:

$$\frac{160}{115}\times100$$
=
139.13

For Year 2:

$$\frac{200}{143.75}\times100$$
=
139.13

Q4 Seasonal Relative

```
Q4 = sales_data$Q4 /
sales_data$Annual_Average * 100
```

For Year 1:

$$\frac{100}{115}\times100$$
$$=$$
$$86.96$$

For Year 2:

$$\frac{125}{143.75}\times100$$
$$=$$
$$86.96$$

The resulting table is:

Quarter	Year 1	Year 2
Q1	69.57	69.57
Q2	104.35	104.35
Q3	139.13	139.13
Q4	86.96	86.96

---

10. Display Seasonal Relatives

cat("\nSeasonal Relatives:\n")

cat() displays text and values.

\n means new line.

"Seasonal Relatives:" is the heading.

So it prints:

Seasonal Relatives:

Then:

print(seasonal_relative)

prints the seasonal-relative table.

---

11. Step 4 — Calculate Seasonal Indices

```
seasonal_index <-
colMeans(seasonal_relative)
```

This calculates the average seasonal relative for each quarter.

colMeans() means:

> Calculate the mean/average of each column.

For Q1:

$$\frac{69.57+69.57}{2}=69.57$$

For Q2:

$$\frac{104.35+104.35}{2}=104.35$$

For Q3:

$$\frac{139.13+139.13}{2}=139.13$$

For Q4:

$$\frac{86.96+86.96}{2}=86.96$$

Therefore:

```
Q1     69.57
Q2    104.35
Q3    139.13
Q4     86.96
```

These are the seasonal indices.

---

12. Display Seasonal Indices

```
cat("\nSeasonal Indices:\n")
```

Prints the heading:

```
Seasonal Indices:
```

Then:

```
print(seasonal_index)
```

displays the calculated indices.

---

13. Step 5 — Calculate Year 3 Trend

year1_sales <- sales_data$Annual_Sales[1]

sales_data$Annual_Sales accesses the Annual Sales column.

[1] selects the first row.

Therefore:

year1_sales=460

---

year2_sales <- sales_data$Annual_Sales[2]

[2] selects the second row.

Therefore:

year2_sales=575

---

14. Calculate Annual Increase

annual_increase <- year2_sales - year1_sales

Calculates:

575-460=115

Therefore:

\boxed{Annual\ Increase=115\text{ lakh}}

---

15. Forecast Year 3 Annual Sales

year3_sales <- year2_sales + annual_increase

Assuming the same trend continues:

575+115=690

Therefore:

$$\boxed{Year\ 3=690\text{ lakh}}$$

---

16. Display Trend Calculations

```
cat("\nYear 1 Sales =", year1_sales,
"lakh\n")
```

Displays:

Year 1 Sales = 460 lakh

```
cat("Year 2 Sales =", year2_sales,
"lakh\n")
```

Displays:

Year 2 Sales = 575 lakh

```
cat("Annual Increase =", annual_increase,
"lakh\n")
```

Displays:

```
Annual Increase = 115 lakh
```

```
cat("Forecasted Year 3 Sales =",
year3_sales, "lakh\n")
```

Displays:

```
Forecasted Year 3 Sales = 690 lakh
```

---

17. Step 6 — Calculate Average Quarterly Sales

```
year3_quarterly_average <- year3_sales /
4
```

Year 3 total forecast is ₹690 lakh.

Since there are four quarters:

$$\frac{690}{4}=172.5$$

Therefore:

$$\boxed{Average\ Quarterly\ Sales=172.5\text{ lakh}}$$

---

18. Display Average Quarterly Sales

```
cat(
  "Year 3 Average Quarterly Sales =",
  year3_quarterly_average,
  "lakh\n"
)
```

This displays:

Year 3 Average Quarterly Sales = 172.5 lakh

The arguments of cat() are:

1. Text

2. Calculated value

3. Unit

4. New line

---

19. Step 7 — Forecast Q1

```
forecast_Q1 <- year3_quarterly_average *
  seasonal_index["Q1"] / 100
```

The formula is:

$$
\text{Forecast} = \text{Trend\ Average} \times \frac{\text{Seasonal\ Index}}{100}
$$

For Q1:

```
172.5\times\frac{69.57}{100}

\approx120

Therefore:

\boxed{Q1=120\text{ lakh}}


---

20. Forecast Q2

forecast_Q2 <- year3_quarterly_average *
  seasonal_index["Q2"] / 100

Calculates:

172.5\times\frac{104.35}{100}

\approx180

Therefore:

\boxed{Q2=180\text{ lakh}}
```

---

21. Forecast Q3

```
forecast_Q3 <- year3_quarterly_average *
  seasonal_index["Q3"] / 100
```

Calculates:

$$172.5\times\frac{139.13}{100}$$

$$\approx240$$

Therefore:

$$\boxed{Q3=240\text{ lakh}}$$

---

22. Forecast Q4

```
forecast_Q4 <- year3_quarterly_average *
  seasonal_index["Q4"] / 100
```

Calculates:

$$172.5\times\frac{86.96}{100}$$

\approx150

Therefore:

\boxed{Q4=150\text{ lakh}}

---

23. Step 8 — Create Forecast Table

forecast <- data.frame(

Creates a new data frame called forecast.

Quarter = c("Q1", "Q2", "Q3", "Q4"),

Creates the Quarter column:

Q1
Q2
Q3
Q4

```
Seasonal_Index =
as.numeric(seasonal_index),
```

seasonal_index contains the seasonal indices.

as.numeric() converts the values into numeric form.

This creates the Seasonal_Index column.

```
Forecast_Sales = c(
  forecast_Q1,
  forecast_Q2,
  forecast_Q3,
  forecast_Q4
)
```

Creates a column called Forecast_Sales.

It contains:

120
180
240
150

Therefore the final table is:

Quarter Seasonal Index Forecast Sales

Q1 69.57 120
Q2 104.35 180
Q3 139.13 240
Q4 86.96 150

---

24. Display Forecast

cat("\nYear 3 Sales Forecast:\n")

Displays:

Year 3 Sales Forecast:

Then:

```
print(forecast)
```

displays the complete forecast table.

---

25. Step 9 — Calculate Total Forecast

```
total_forecast <-
sum(forecast$Forecast_Sales)
```

forecast$Forecast_Sales accesses the Forecast Sales column.

sum() adds all values.

Therefore:

$$120+180+240+150=690$$

So:

$$\boxed{Total\ Forecast=690\text{ lakh}}$$

---

26. Display Total Forecast

```
cat(
  "\nTotal Forecasted Sales for Year 3
=",
  total_forecast,
  "lakh\n"
)
```

Displays:

```
Total Forecasted Sales for Year 3 = 690
lakh
```

Complete Statistical Flow

The entire program follows this sequence:

```
Historical Sales
       ↓
Annual Sales
       ↓
Annual Average
```

↓

Seasonal Relatives

↓

Seasonal Indices

↓

Year 3 Trend

↓

Average Quarterly Trend

↓

Apply Seasonal Indices

↓

Year 3 Quarterly Forecast

↓

Total Forecast

Final result

Quarter	Seasonal Index	Year 3 Forecast
Q1	69.57%	₹120 lakh
Q2	104.35%	₹180 lakh
Q3	139.13%	₹240 lakh
Q4	86.96%	₹150 lakh
Total	400%	₹690 lakh

The key statistical idea is that the trend estimates the overall level of

future sales, while the seasonal index adjusts that level according to the normal seasonal pattern of each quarter.

Practical 7

Index numbers - basics and applications

```
#
=============================================
====================
# INDEX NUMBERS - BASICS AND APPLICATIONS
# CASE STUDY: COST OF LIVING INDEX
#
=============================================
====================

#
---------------------------------------------
--------------------
# 1. CLEAR ENVIRONMENT
#
---------------------------------------------
--------------------

rm(list = ls())

# Close existing graphics devices safely
while (!is.null(dev.list())) {
  dev.off()
}
```

```
#
-------------------------------------------
--------------------
# 2. CREATE DATA
#
-------------------------------------------
--------------------

commodity <- c(
  "Rice",
  "Wheat",
  "Milk",
  "Sugar",
  "Cooking Oil"
)

# Base-year quantity
q0 <- c(20, 15, 30, 10, 8)

# Current-year quantity
q1 <- c(22, 16, 32, 11, 8)

# Base-year price (2024)
p0 <- c(50, 40, 50, 45, 120)

# Current-year price (2026)
p1 <- c(60, 48, 60, 54, 150)
```

```
#
-------------------------------------------
-------------------
# 3. CREATE DATA FRAME
#
-------------------------------------------
-------------------

index_data <- data.frame(
  Commodity = commodity,
  Base_Quantity = q0,
  Current_Quantity = q1,
  Base_Price = p0,
  Current_Price = p1
)

cat("\n================ ORIGINAL DATA
================\n")

print(index_data)


#
-------------------------------------------
-------------------
# 4. CALCULATE EXPENDITURES
```

```
#
-----------------------------------------
------------------
```

```
# P0Q0 = Base Price × Base Quantity
index_data$P0Q0 <-
  index_data$Base_Price *
  index_data$Base_Quantity
```

```
# P1Q0 = Current Price × Base Quantity
index_data$P1Q0 <-
  index_data$Current_Price *
  index_data$Base_Quantity
```

```
# P0Q1 = Base Price × Current Quantity
index_data$P0Q1 <-
  index_data$Base_Price *
  index_data$Current_Quantity
```

```
# P1Q1 = Current Price × Current Quantity
index_data$P1Q1 <-
  index_data$Current_Price *
  index_data$Current_Quantity
```

```
cat("\n================ EXPENDITURE TABLE
================\n")
```

```
print(index_data)


#
-------------------------------------------
-------------------
# 5. PRICE RELATIVES
#
-------------------------------------------
-------------------

# Formula:
# Price Relative = (P1 / P0) × 100

index_data$Price_Relative <-
  (index_data$Current_Price /
     index_data$Base_Price) * 100


cat("\n================ PRICE RELATIVES
================\n")

print(
  index_data[
    c("Commodity", "Price_Relative")
  ]
)
```

```
#
-------------------------------------------
--------------------
# 6. SIMPLE AGGREGATE PRICE INDEX
#
-------------------------------------------
--------------------

# Formula:
#
# Simple Price Index =
# (ΣP1 / ΣP0) × 100

simple_index <-
  (sum(p1) / sum(p0)) * 100


#
-------------------------------------------
--------------------
# 7. SIMPLE AVERAGE OF PRICE RELATIVES
#
-------------------------------------------
--------------------

# Formula:
#
```

```
# Average Price Relative =
# Σ(P1/P0 × 100) / Number of commodities

average_index <-
  mean(index_data$Price_Relative)


#
-------------------------------------------
--------------------
# 8. LASPEYRES PRICE INDEX
#
-------------------------------------------
--------------------

# Formula:
#
# L = (ΣP1Q0 / ΣP0Q0) × 100

laspeyres_index <-
  (sum(index_data$P1Q0) /
     sum(index_data$P0Q0)) * 100


#
-------------------------------------------
--------------------
# 9. PAASCHE PRICE INDEX
```

```
#
-------------------------------------------
--------------------

# Formula:
#
# P = (ΣP1Q1 / ΣP0Q1) × 100

paasche_index <-
  (sum(index_data$P1Q1) /
     sum(index_data$P0Q1)) * 100


#
-------------------------------------------
--------------------
# 10. FISHER'S IDEAL PRICE INDEX
#
-------------------------------------------
--------------------

# Formula:
#
# Fisher = √(Laspeyres × Paasche)

fisher_index <-
  sqrt(laspeyres_index * paasche_index)
```

```
#
-----------------------------------------
-------------------
# 11. TOTAL COST OF CONSUMPTION BASKET
#
-----------------------------------------
-------------------

base_cost <-
  sum(index_data$P0Q0)

current_cost <-
  sum(index_data$P1Q1)


#
-----------------------------------------
-------------------
# 12. PERCENTAGE CHANGE IN COST
#
-----------------------------------------
-------------------

percentage_change <-
  ((current_cost - base_cost) /
     base_cost) * 100
```

```
#
-------------------------------------------
-------------------
# 13. DISPLAY RESULTS
#
-------------------------------------------
-------------------

cat("\n\n================================
====================")
cat("\n                    INDEX NUMBER
RESULTS")
cat("\n================================
==================\n")

cat(
  "\nSimple Aggregate Price Index : ",
  round(simple_index, 2)
)

cat(
  "\nAverage Price Relative       : ",
  round(average_index, 2)
)

cat(
  "\nLaspeyres Price Index        : ",
```

```
  round(laspeyres_index, 2)
)

cat(
  "\nPaasche Price Index              : ",
  round(paasche_index, 2)
)

cat(
  "\nFisher's Ideal Price Index    : ",
  round(fisher_index, 2)
)

cat(
  "\n\nBase Year Basket Cost          : ₹",
  base_cost
)

cat(
  "\nCurrent Year Basket Cost      : ₹",
  current_cost
)

cat(
  "\nPercentage Increase in Cost  : ",
  round(percentage_change, 2),
  "%"
)
```

```
#
-------------------------------------------
-------------------
# 14. RESULT TABLE
#
-------------------------------------------
-------------------

results <- data.frame(
  Method = c(
    "Simple Aggregate",
    "Average Price Relative",
    "Laspeyres",
    "Paasche",
    "Fisher's Ideal"
  ),

  Index = round(
    c(
      simple_index,
      average_index,
      laspeyres_index,
      paasche_index,
      fisher_index
    ),
    2
```

```
  )
)


cat("\n\n================ RESULT TABLE
================\n")

print(results)


#
-----------------------------------------
--------------------
# 15. INTERPRETATION
#
-----------------------------------------
--------------------

cat("\n\n================ INTERPRETATION
================\n")

if (fisher_index > 100) {

  cat(
    "\nThe overall price level has
increased ",
    "compared with the base year."
  )
```

```
} else if (fisher_index < 100) {

  cat(
    "\nThe overall price level has
decreased ",
    "compared with the base year."
  )

} else {

  cat(
    "\nThere is no overall change in the
price level."
  )
}


cat(
  "\n\nFisher's Index indicates that the
price level is ",
  round(fisher_index, 2),
  " compared with the base year index of
100."
)
```

```
#
==========================================
==========================
# 16. GRAPH 1
# BASE YEAR VS CURRENT YEAR PRICES
#
==========================================
==========================

# Reset plotting parameters
par(mfrow = c(1, 1))

# Increase bottom margin for commodity
names
par(
  mar = c(8, 5, 4, 2) + 0.1
)

barplot(
  rbind(p0, p1),

  beside = TRUE,

  names.arg = commodity,

  main = "Base Year vs Current Year
Prices",
```

```
  xlab = "Commodity",

  ylab = "Price (Rs.)",

  las = 2,

  cex.names = 0.8,

  legend.text = c(
    "Base Year (2024)",
    "Current Year (2026)"
  )
)


#
==========================================
==================
# 17. GRAPH 2
# COMPARISON OF INDEX NUMBERS
#
==========================================
==================

# Reset margins
par(
  mar = c(6, 5, 4, 2) + 0.1
)
```

```
index_values <- c(
  simple_index,
  average_index,
  laspeyres_index,
  paasche_index,
  fisher_index
)

index_names <- c(
  "Simple",
  "Average",
  "Laspeyres",
  "Paasche",
  "Fisher"
)

barplot(
  index_values,

  names.arg = index_names,

  main = "Comparison of Index Numbers",

  xlab = "Index Method",

  ylab = "Index Value",
```

```
  ylim = c(
    0,
    max(index_values) + 10
  ),

  cex.names = 0.8
)

# Base index line = 100
abline(
  h = 100,
  lty = 2
)


#
============================================
====================
# 18. FINAL MESSAGE
#
============================================
====================

cat("\n\n============================================
====================")
cat("\n                        CASE STUDY
COMPLETE")
```

```
cat("\n==========================================
==================")

cat(
  "\n\nBase Year: 2024"
)

cat(
  "\nCurrent Year: 2026"
)

cat(
  "\n\nThe R program successfully
calculated:"
)

cat(
  "\n1. Simple Aggregate Price Index"
)

cat(
  "\n2. Average Price Relative"
)

cat(
  "\n3. Laspeyres Price Index"
)
```

```
cat(
  "\n4. Paasche Price Index"
)

cat(
  "\n5. Fisher's Ideal Price Index"
)

cat(
  "\n6. Cost of Consumption Basket"
)

cat(
  "\n7. Percentage Change"
)

cat("\n")
```

Output

```
================ ORIGINAL DATA
================

    Commodity Base_Quantity
Current_Quantity Base_Price Current_Price
1         Rice             20
22           50             60
2        Wheat             15
16           40             48
3         Milk             30
32           50             60
4        Sugar             10
11           45             54
5 Cooking Oil              8
8          120           150


================ EXPENDITURE TABLE
================

    Commodity Base_Quantity
Current_Quantity Base_Price Current_Price
P0Q0 P1Q0 P0Q1 P1Q1
1         Rice             20
22           50             60 1000 1200
1100 1320
```

```
2        Wheat              15
16          40              48   600   720
640   768
3         Milk              30
32          50              60  1500  1800
1600  1920
4        Sugar              10
11          45              54   450   540
495   594
5 Cooking Oil                8
8          120             150   960  1200   960
1200
```

========== PRICE RELATIVES ==========

	Commodity	Price_Relative
1	Rice	120
2	Wheat	120
3	Milk	120
4	Sugar	120
5	Cooking Oil	125

==========

INDEX NUMBER RESULTS

```
==========================================
==========

Simple Aggregate Price Index :  121.97
Average Price Relative       :  121
Laspeyres Price Index        :  121.06
Paasche Price Index          :  121
Fisher's Ideal Price Index   :  121.03

Base Year Basket Cost        : ₹ 4510
Current Year Basket Cost     : ₹ 5802
Percentage Increase in Cost  :  28.65 %


================ RESULT TABLE
================

                  Method  Index
1       Simple Aggregate 121.97
2 Average Price Relative 121.00
3              Laspeyres 121.06
4                Paasche 121.00
5         Fisher's Ideal 121.03


================ INTERPRETATION
================
```

The overall price level has increased compared with the base year.

Fisher's Index indicates that the price level is 121.03 compared with the base year index of 100.

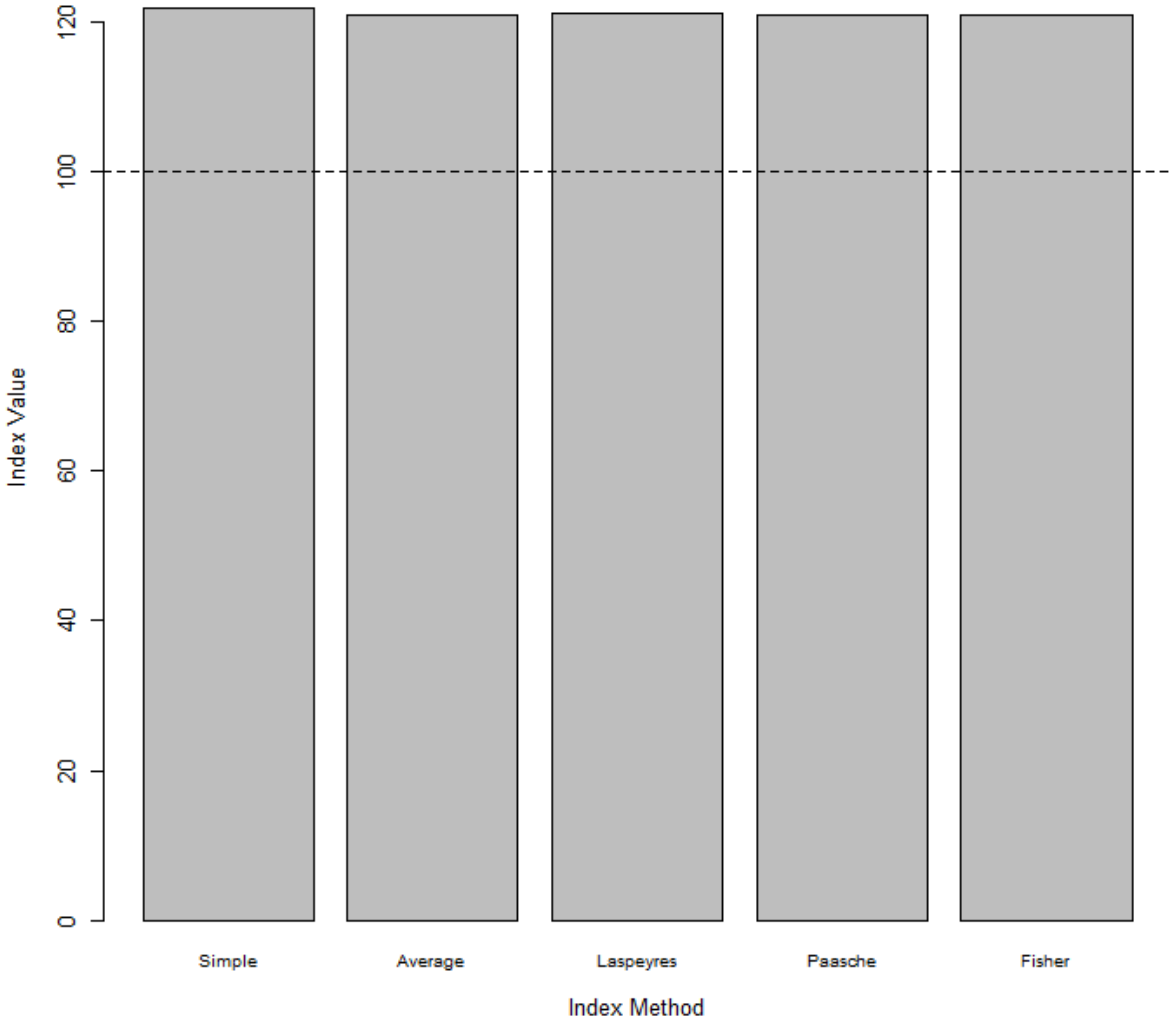
==========

CASE STUDY COMPLETE

==========

Base Year: 2024
Current Year: 2026

The R program successfully calculated:
1. Simple Aggregate Price Index
2. Average Price Relative
3. Laspeyres Price Index
4. Paasche Price Index
5. Fisher's Ideal Price Index
6. Cost of Consumption Basket
7. Percentage Change


Comparison of Index Numbers
Index Value
0
20
40
60
80
100
120
Simple
Average
Laspeyres
Paasche
Fisher
Index Method